주행 상황 실시간 경고 시스템

별첨 문서집

Supplementary Appendix Bundle

본 문서집은 소논문 본문을 보강하기 위한 안전, 계약, 검증, 진단 문서를 묶어 정리한 reviewer-facing 별첨 자료이다.

Project	Driving Alert Project
Bundle Scope	Governance / Runtime Contracts / Verification / Diagnostic
Language	Korean reviewer-facing edition
Source Rule	Canonical source paths are preserved in each supplementary note

Workspace: driving-alert-workproducts/governance/short-paper/appendix

Generated from local source copies under appendix/source/

Contents

S1. 안전 및 추적성 기준 **5**
0.1 HARA 워크시트 (Hazard Analysis and Risk Assessment) . . . 9
0.1.1 1. 목적 및 범위 . . . 9
0.1.2 2. S/E/C 평가 기준(요약) . . . 9
0.1.3 3. HARA 요약 워크시트 . . . 9
0.2 ECU 명명 및 계층 표준 . . . 10
0.2.1 1. 목적 . . . 10
0.2.2 2. 기본 원칙 . . . 10
0.2.3 3. 표기 규칙 . . . 10
0.2.4 4. Canonical 약어 기준 . . . 10
0.2.5 5. 주요 표면 ECU 기준 . . . 10
0.2.6 6. 문서 적용 기준 . . . 11
0.3 CAN ID 배정 표준 . . . 12
0.3.1 1. 현재 운영 구조 . . . 12
0.3.2 1-1. Ethernet 사전 정의 (나중에 이더넷 실 전환) . . . 12
0.3.3 2. 목적 . . . 12
0.3.4 3. 배정 원칙 . . . 12
0.3.5 4. CAN Active ID 대역 (11-bit) . . . 12
0.3.6 4-1. Ethernet/UDP 식별자 운영 값 . . . 13
0.3.7 5. 예약 구간 (식별자 정책) . . . 13
0.3.8 6. 배정 절차 . . . 13
0.3.9 7. 운영 메모 . . . 13
0.4 Master Test Matrix . . . 14
0.4.1 1. 목적 . . . 14
0.4.2 2. 컬럼 운영 원칙 . . . 14
0.4.3 3. 01 문서와의 관계 . . . 14
0.4.4 4. Master Test Matrix (Draft Baseline) . . . 14
0.4.5 5. 운영 규칙 . . . 17
0.4.6 6. 다음 단계 . . . 18

S2. 런타임 계약 세트 **19**
0.5 Runtime Message Ownership Matrix . . . 21
0.5.1 특수 해석 규칙 . . . 21
0.6 Owner / Bus / Timeout / Route Contract . . . 27
0.6.1 목적 . . . 27
0.6.2 계약 차원 . . . 27
0.6.3 규칙 . . . 27
0.6.4 현재 seam 표 . . . 27
0.6.5 사용 규칙 . . . 28
0.6.6 개발 메모 . . . 28
0.7 Runtime Layer Separation Policy . . . 29
0.7.1 목적 . . . 29
0.7.2 외부 기준 . . . 29
0.7.3 계층 모델 . . . 29
0.7.4 분리 규칙 . . . 30
0.7.5 현재 프로젝트 적용 . . . 30
0.7.6 현재 backbone producer boundary 규칙 . . . 31
0.7.7 읽는 순서 . . . 32
0.7.8 갱신 규칙 . . . 32
0.8 ETH Interface Contract . . . 33

0.8.1 1. 목적 . . . 33
0.8.2 2. Active transport baseline . . . 33
0.8.3 3. Contract seam 분류 . . . 33
0.8.4 4. 핵심 규칙 . . . 34
0.8.5 5. 사용 규칙 . . . 35
0.9 Multi-Bus Visibility Policy . . . 36
0.9.1 1. 목적 . . . 36
0.9.2 2. 핵심 원칙 . . . 36
0.9.3 3. GUI 해석 규칙 . . . 36
0.9.4 4. Multibus가 필요한 이유 . . . 36
0.9.5 5. Node 분류 . . . 37
0.9.6 6. TEST_BAS 단일-bus 유지 규칙 . . . 38
0.9.7 7. 사용 규칙 . . . 38
0.10 Panel and SysVar Binding Contract . . . 39
0.10.1 1. 목적 . . . 39
0.10.2 2. Active binding 원칙 . . . 39
0.10.3 3. Write surface . . . 39
0.10.4 4. Read surface . . . 40
0.10.5 5. Binding 규칙 . . . 41
0.11 Ethernet Backbone Interface Specification 안내본 . . . 42
0.11.1 이 문서에서 확인할 것 . . . 42
0.11.2 바로 가기 . . . 42

S3. 검증 계약 세트 **43**
0.12 Oracle . . . 43
0.12.1 목적 . . . 43
0.12.2 Oracle 모델 . . . 43
0.12.3 Oracle source . . . 43
0.12.4 현재 판정 규칙 . . . 44
0.12.5 Harness 기반 oracle 해석 . . . 44
0.12.6 현재 oracle 경계 . . . 44
0.12.7 개발 메모 . . . 44
0.13 Acceptance Criteria . . . 45
0.13.1 목적 . . . 45
0.13.2 Scenario verdict 표 . . . 45
0.13.3 Verdict 흐름 . . . 45
0.13.4 현재 oracle signal . . . 46
0.13.5 Output-style reference . . . 46
0.13.6 개발 메모 . . . 46
0.14 Test Asset Mapping . . . 47
0.14.1 1. 목적 . . . 47
0.14.2 2. 매핑 규칙 . . . 47
0.14.3 3. Unit Test 매핑 . . . 47
0.14.4 4. Integration Test 매핑 . . . 50
0.14.5 5. System Test 매핑 . . . 51
0.14.6 6. 현재 구현 우선순위 . . . 52
0.14.7 7. 다른 verification 문서와의 관계 . . . 53
0.14.8 최신 diagnostic 실행 기준선 (2026-03-15) . . . 53
0.14.9 Wave 2 direct-ownership UT 기준선 . . . 53
0.14.10 Wave 2 oracle 기준선 . . . 53
0.14.11 Wave 2 현재 진행 업데이트 . . . 53
0.14.12 Wave 2 진행 업데이트 (208/209) . . . 54

0.14.13 External TX unit 완료 업데이트 . . . 54
0.14.14 Wave 3 system-test 기준선 . . . 54
0.14.15 Wave 2 완료 업데이트 (UT_004/UT_005) . . . 55
0.14.16 Wave 5 gateway 및 extension UT 기준선 . . . 56
0.14.17 Wave 6 service, assist, router UT 기준선 . . . 56
0.14.18 Wave 4 integration 기준선 . . . 56
0.15 실행 가이드 . . . 58
0.15.1 목적 . . . 58
0.15.2 검증 모델 . . . 58
0.15.3 현재 active native asset . . . 58
0.15.4 Harness contract . . . 58
0.15.5 표준 실행 절차 . . . 59
0.15.6 현재 매핑 . . . 59
0.15.7 기대 산출물 . . . 59
0.15.8 경계 . . . 59
0.15.9 개발 메모 . . . 59
0.16 검증 증빙 로그 표준 (CANoe SIL) . . . 60
0.16.1 1) 목적 . . . 60
0.16.2 2) 고정 실행 순서 . . . 60
0.16.3 3) 개발완성도 체크 기준 (Step 1) . . . 60
0.16.4 4) 증빙 폴더 규칙 . . . 60
0.16.5 5) verification_log.csv 필수 컬럼 . . . 61
0.16.6 6) 판정 규칙 . . . 62
0.16.7 7) 표준 시간 기준 . . . 63
0.16.8 8) 캡처 규칙 . . . 63
0.16.9 9) 실행 명령 예시 . . . 63
0.16.10 10) 05/06/07 반영 규칙 . . . 63
0.16.11 11) 승인 규칙 . . . 63

S4. 진단 및 관측성 세트 **64**
0.17 Diagnostic Matrix . . . 64
0.17.1 1. 목적 . . . 64
0.17.2 2. 최소 matrix 필드 . . . 64
0.17.3 3. 현재 공식 matrix . . . 65
0.17.4 4. 필드 해석 규칙 . . . 65
0.17.5 5. 향후 심화 필드 . . . 65
0.17.6 6. 관련 문서 . . . 65
0.18 Diagnostic SysVar Contract . . . 66
0.18.1 1. 목적 . . . 66
0.18.2 2. 범위 . . . 66
0.18.3 3. Namespace baseline . . . 66
0.18.4 4. Request-side contract . . . 66
0.18.5 5. Response-side contract . . . 66
0.18.6 6. Verdict-facing seam contract . . . 67
0.18.7 7. 계약 규칙 . . . 67
0.18.8 8. Verification 사용 방식 . . . 68
0.18.9 9. 갱신 규칙 . . . 68
0.19 Diagnostic Seam Design . . . 69
0.19.1 1. 목적 . . . 69
0.19.2 2. 설계 원칙 . . . 69
0.19.3 3. 최소 seam field . . . 69
0.19.4 4. 현재 0304-aligned seam candidate . . . 70

0.19.5 5. 현재 reserved seam addition 70
0.19.6 6. 남아 있는 meaning-level gap 70
0.19.7 7. Baseline enum contract 71
0.19.8 8. 현재 official scope별 seam design 71
0.19.9 9. Planned assertion profile 71
0.19.10 10. Publisher ownership baseline 72
0.19.11 11. ST_043 scenario stimulus contract 72
0.19.12 12. Response interpretation baseline 72
0.19.13 13. 구현 순서 72
0.19.14 14. Expansion gate 73
0.19.15 15. 관련 verification 문서 73

S1. 안전 및 추적성 기준

본 별첨은 안전 목표, 위험 사건, ECU naming, CAN ID allocation, Test Matrix를 함께 제시하여 프로젝트의 안전성과 검증 추적성을 보강한다.

원문 기준 경로

- appendix/source/governance/project_v_model_traceability_map.md
- appendix/source/governance/project_v_model_traceability_map.puml
- driving-alert-workproducts/governance/00d_HARA_Worksheet.md
- driving-alert-workproducts/governance/00e_ECU_Naming_Standard.md
- driving-alert-workproducts/governance/00f_CAN_ID_Allocation_Standard.md
- driving-alert-workproducts/governance/00g_Master_Test_Matrix.md

프로젝트 V-모델과 추적 구조

본 별첨의 V-모델 도식은 이론적 일반 모델이 아니라, 현재 active baseline에서 실제로 운영 중인 문서 체계와 검증 체계를 기준으로 정리하였다. 특히 00d_HARA_Worksheet, 01_Requirements, 03 / 0301 / 0302, 0303 / 0304, 04_SW_Implementation, 05 / 06 / 07의 연결 관계와 00g_Master_Test_Matrix의 traceability 역할을 한 장에서 읽을 수 있도록 구성하였다.

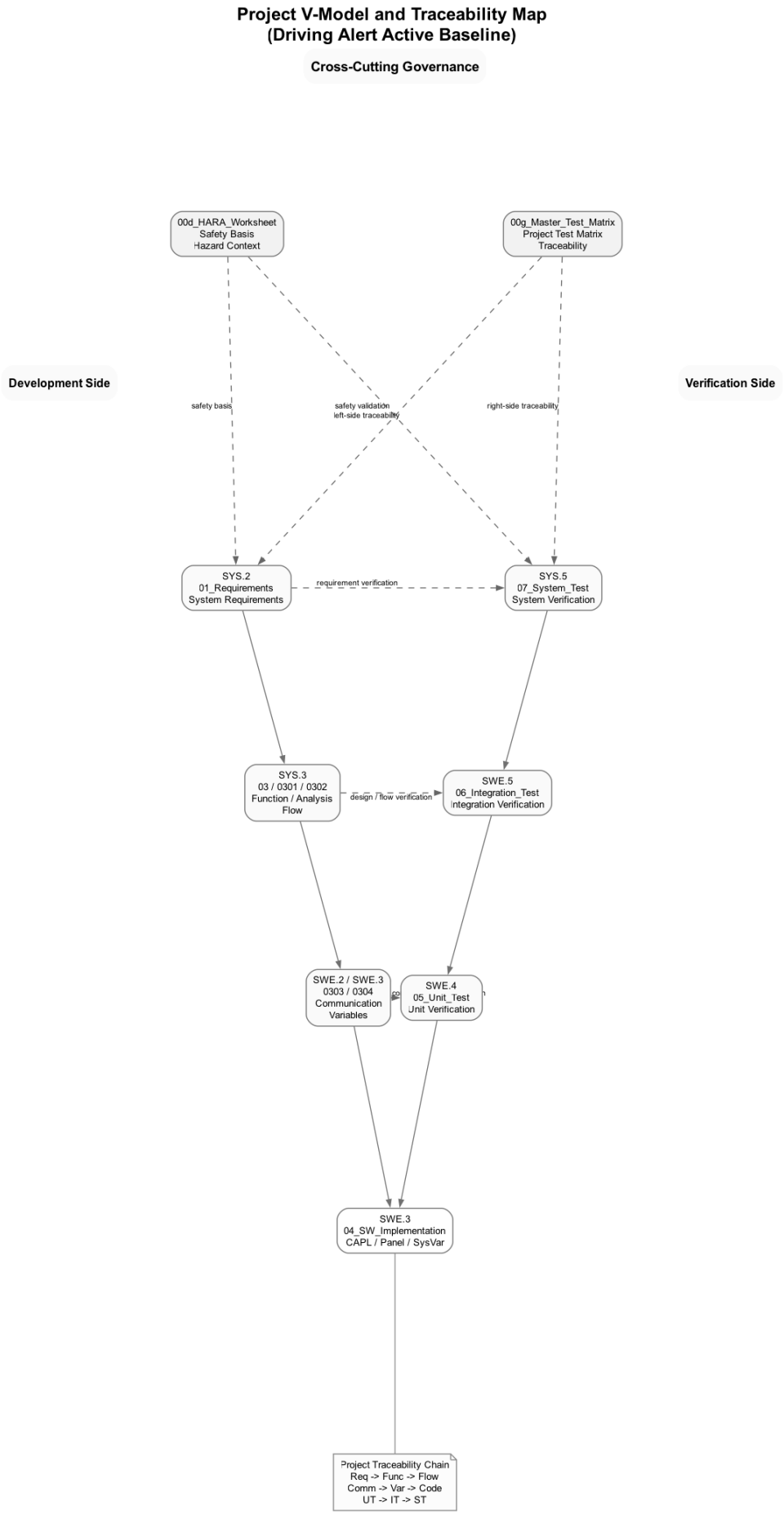

Figure 1: Project V-model and traceability map. Development-side documents, verification-side documents, and cross-cutting governance documents are arranged according to the active baseline used in this project.

프로젝트 V-모델 및 추적 구조

본 문서는 주행 상황 실시간 경고 시스템 프로젝트의 개발 문서와 검증 문서가 실제 V-모델 구조 안에서 어떻게 대응되는지를 reviewer-facing 형식으로 정리한 별첨이다.

본 프로젝트는 CANoe SIL 기반 구현형 프로젝트로, ASPICE 전체 산출물을 모두 독립 문서로 분리하지는 않았다. 대신 01 → 03 → 0301 → 0302 → 0303 → 0304 → 04 → 05 → 06 → 07 문서 체인을 실행 가능한 기준선으로 유지하고, 00d_HARA_Worksheet와 00g_Master_Test_Matrix를 통해 안전성 및 추적성을 고정하였다.

프로젝트 V-모델 해석

- 좌측 개발 축은 요구사항, 시스템 설계, 통신/변수 계약, CAPL 구현 순으로 구성하였다.
- 우측 검증 축은 단위시험, 통합시험, 시스템시험 순으로 구성하였다.
- 00d_HARA_Worksheet는 안전 사건과 검증 우선순위의 기준점으로 사용하였다.
- 00g_Master_Test_Matrix는 요구사항, 기능, 시험 자산을 연결하는 프로젝트 Test Matrix로 사용하였다.

V-모델 매핑 표

관점	ASPICE / ISO 26262	프로젝트 문서	검증 대응
안전 / 추적성 기준	SYS.2, SUP.10 / Part 3, Part 8	00d_HARA_Worksheet, 00g_Master_Test_Matrix	05, 06, 07 전 계층
시스템 요구사항	SYS.2 / Part 4, Cl.6	01_Requirements	07_System_Te
시스템 설계	SYS.3 / Part 4, Cl.7	03, 0301, 0302	06, 07
통신 / 변수 계약	SWE.2, SWE.3 / Part 6, Cl.7–8	0303, 0304	05, 06
구현 기준선	SWE.3 / Part 6, Cl.9	04_SW_Implementation	05, 06
단위 검증	SWE.4 / Part 6, Cl.9	05_Unit_Test	0303, 0304, 04
통합 검증	SWE.5 / Part 6, Cl.10	06_Integration_Test	03, 0301, 0302, 0303, 0304
시스템 검증	SYS.5 / Part 4, Cl.10	07_System_Test	01_Requiremen 00d_HARA_W

표의 해석

- 00d_HARA_Worksheet는 위험 사건과 안전 우선순위의 기준점이다.
- 00g_Master_Test_Matrix는 요구사항, 기능, 시험 자산을 연결하는 프로젝트 Test Matrix이다.
- 03, 0301, 0302는 ECU 역할, 기능 분해, 네트워크 흐름을 담당하는 시스템 설계 축이다.
- 0303, 0304는 실행 가능한 통신/변수 계약을 정의하는 인터페이스 축이다.
- 04_SW_Implementation은 CAPL, panel, sysvar를 포함한 구현 기준선이다.

프로젝트 추적성 체인 본 프로젝트의 추적성 체인은 아래 순서로 고정하였다.

Req → Func → Flow → Comm → Var → Code → Unit Test → Integration Test → System Test

작성 기준

- 본 매핑은 현재 active baseline 문서를 기준으로 정리하였다.
- 별도 SWE.1 독립 문서는 두지 않고, 기능 정의와 통신/변수 계약 문서가 소프트웨어 요구/설계 역할을 분담하도록 구성하였다.
- 따라서 본 프로젝트의 V-모델은 이론적 완전 분리형이 아니라, CANoe SIL 구현 프로젝트에 맞춘 실행형 추적 구조로 해석한다.

0.1 HARA 워크시트 (Hazard Analysis and Risk Assessment)

Document ID: PROJ-00D-HARA **ISO 26262 Reference**: Part 3 (Concept Phase, Hazard Analysis and Risk Assessment) **ASPICE Reference**: SYS.2, SUP.10 **Version**: 1.6 **Date**: 2026-03-11 **Status**: Draft (Submission Summary) **Project Title**: 주행 상황 실시간 경고 시스템 **Subtitle**: 구간 정보 및 긴급차량 접근 기반 앰비언트·클러스터 경보

제출용 정리본: 주요 위험 이벤트, 안전 목표, 대표 검증 근거만 유지한 문서입니다.

0.1.1 1. 목적 및 범위

본 문서는 프로젝트 주요 위험 상황에 대한 S/E/C 평가, ASIL 후보, 안전 목표, 대표 검증 관점을 정리한다.

0.1.2 2. S/E/C 평가 기준(요약)

항목	레벨	의미
Severity (S)	S0~S3	부상 및 사고 위험의 심각도
Exposure (E)	E0~E4	해당 상황에 노출될 가능성
Controllability (C)	C0~C3	운전자가 상황을 제어할 수 있는 정도

0.1.3 3. HARA 요약 워크시트

HARA ID	관련 Req	Hazardous Event (요약)
HC-01	Req_010	스쿨존 과속 경고 누락 또는 지연으로 감속 시점을 놓
HC-02	Req_011, Req_012	고속 구간 무조향 의심 경고가 발생하지 않거나 해제
HC-03	Req_021, Req_024, Req_026~Req_031	긴급 경고와 구간 경고가 충돌할 때 우선순위가 잘못
HC-04	Req_023, Req_032, Req_033	타임아웃 또는 복귀 전환이 불안정해 경고가 반복 표
HC-05	Req_054~Req_056	경고 전달 경계 또는 게이트웨이 전달 오류로 경고가
HC-06	Req_057~Req_066	객체 인지 또는 위험 판단 실패로 위험 경고가 누락되
HC-07	Req_075~Req_079	입력 stale 또는 채널 장애 시 경고 연속성과 강등 출력
HC-08	Req_080~Req_082	오디오 경합, 팝업 과밀, 채널 비동기로 경고 인지가

0.2 ECU 명명 및 계층 표준

Document ID: PROJ-00E-ECU-NAMING **Version**: 3.9 **Date**: 2026-03-09 **Status**: Draft (Submission Summary) **Scope**: 00e -> 0301 -> 0302 -> 0303 -> 0304 -> 04 -> 05 -> 06 -> 07

제출용 정리본: 원본 SoT에서 문서 표기에 필요한 명명 규칙과 주요 표면 ECU만 유지한 문서입니다.

0.2.1 1. 목적

본 문서는 제출 문서에서 사용할 ECU 표면 이름과 기본 명명 규칙을 정리한다. 상위 문서에서는 차량 관점의 표면 ECU 이름을 사용하고, 구현 상세 이름은 필요한 경우에만 괄호로 병기한다.

0.2.2 2. 기본 원칙

- 상위 문서에서는 표면 ECU 이름을 우선 사용한다.
- 구현 상세 이름은 상위 문서의 기본 표기로 사용하지 않는다.
- Validation Harness는 제품 기능 ECU와 분리해 관리한다.
- 문서 전반에서는 국내 OEM 관행 약어형을 Canonical로 사용한다.

0.2.3 3. 표기 규칙

항목	규칙
Surface ECU	EMS, VCU, ESC, MDPS, BCM, IVI, CLU, ADAS, V2X, CGW처럼 업계 통용 약어를 사용한다
Runtime Module	_GW, _CTRL, _MGR, _TX, _RX와 같은 구현 이름은 필요 시 괄호로만 병기한다.
Validation	Validation 전용 노드는 production ECU와 섞지 않고 별도로 설명한다.
문서 작성	상단 표와 reviewer-facing 설명은 표면 ECU 기준으로 읽히도록 작성한다.

0.2.4 4. Canonical 약어 기준

Canonical (고정)	Alias (설명용)	비고
SGW	SECURITY_GATEWAY	보안 게이트웨이
_4WD	AWD_4WD	4WD 제어 표면명
DATC	HVAC	공조 제어
AHLS	LIGHTING_ECU	조명 제어
EDR	EDGE_LOGGER	이벤트 기록
EMS	ECM	엔진 제어 표면명
TCU	TCM	변속 제어 표면명
ESC	ESP	제동/차체 제어 표면명
MDPS	EPS	조향 제어 표면명
CLU	CLUSTER	클러스터 표면명
ETHB	ETH_BACKBONE	Ethernet 경계 표면명

0.2.5 5. 주요 표면 ECU 기준

Domain	주요 표면 ECU
Integration	CGW, ETHB, DCM, IBOX, SGW
Powertrain	EMS, TCU, VCU, _4WD, BAT_BMS, OBC, DCDC, MCU, INVERTER
Chassis/Safety	ESC, MDPS, ABS, EPB, EHB, VSM, ECS, CDC, RWS
Body/Comfort	BCM, DATC, AFLS, AHLS, DOOR_FL/FR/RL/RR, TAILGATE_MODULE, SEAT_DRV, SEAT_PASS
IVI/HMI	IVI, CLU, HUD, TMU, AMP, NAV_MODULE, DIGITAL_KEY, RSE
ADAS/V2X	ADAS, V2X, SCC, LDWS_LKAS, FCA, BCW, LCA, SPAS, RSPA, AVM, DMS, OMS

0.2.6 6. 문서 적용 기준

- 0301, 0302, 0303의 상단 공식 표는 표면 ECU 이름을 먼저 쓴다.
- 구현 이름이 필요한 경우에는 괄호 안에만 병기한다.
- 제출 문서에서는 표면 ECU 이름만 읽어도 기능 흐름을 이해할 수 있어야 한다.

0.3 CAN ID 배정 표준

Document ID: PROJ-00F-CAN-ID **Version**: 4.7 **Date**: 2026-03-09 **Status**: Draft (Submission Summary) **Scope**: 0302 -> 0303 -> 0304 -> DBC -> 04 -> 05 -> 06 -> 07

> 제출용 정리본: 현재 실행 기준인 11-bit CAN ID 규칙과 Ethernet/UDP 식별자 규칙을 분리해 정리한 문서입니다.

0.3.1 1. 현재 운영 구조

- 현재 개발 코드와 실행 환경은 CAN/CAN-stub 버스를 11-bit CAN ID로 통합 운영한다.
- 승용차 기준 대역폭, 지연, 운영 단순성을 고려해 현재 단계에서는 11-bit 체계를 유지한다.
- CAN 활성 운영 범위는 0x100 ~ 0x2FF이며, 현재 사용 범위는 0x100 ~ 0x2AA다.
- 0x000 ~ 0x0FF 구간은 신규 할당하지 않는다.
- 진단 및 확장 기능을 위한 29-bit 구조는 별도 확장 설계로 검토 중이며, 현재 실행 기준에는 적용하지 않는다.

0.3.2 1-1. Ethernet 사전 정의 (나중에 이더넷 실 전환)

- CAN ID 제거 가능.
- 대신 아래를 필수 관리:
- 현재 0x510, 0x511, 0x512, 0x206, 0xE100 값은 CAN ID가 아니라 Ethernet/UDP 식별자로 관리한다.
- Protocol (UDP/TCP/SOME-IP/DoIP)
- Service/Event/Method ID (SOME-IP면)
- Src/Dst IP:Port
- PDU/Message Name
- Cycle/Timeout
- Gateway mapping (필요 시 CAN↔ETH 변환 규칙)

0.3.3 2. 목적

본 문서는 CAN 메시지의 ID 배정 원칙과 현재 운영 대역을 정리한다. 문서, DBC, 구현, 시험은 동일한 ID 기준을 사용한다.

0.3.4 3. 배정 원칙

항목	기준
우선순위	차량 거동과 직접 연결되는 제어/상태 메시지를 우선 배정한다.
도메인	Chassis, Powertrain, Body, IVI, ADAS, Backbone 등 Owner 도메인 기준으로 나눈다.
연속성	같은 도메인 메시지는 같은 대역 안에서 연번으로 관리한다.
확장성	신규 메시지를 위해 각 도메인별 예약 구간을 남겨둔다.
동기화	ID 변경 시 0302, 0303, 0304, DBC, 04, 05~07을 함께 갱신한다.

0.3.5 4. CAN Active ID 대역 (11-bit)

ID 대역	용도	대표 영역
0x100 ~ 0x13F	차량 기본 상태, 조향, 제동, 동력 기본 루프	Chassis + Powertrain + 일부 Backbone
0x1C0 ~ 0x20F	긴급 알림, 위험 판단, 주행 보조 상태	ADAS + V2X

ID 대역	용도	대표 영역
0x260 ~ 0x27F	출입, 조명, 공조, 실내 편의 상태	Body
0x280 ~ 0x29F	내비 문맥, 클러스터 표시, HMI 상태	IVI + CLU

0.3.6 4-1. Ethernet/UDP 식별자 운영 값

아래 값은 CAN ID가 아니라 Ethernet/UDP 식별자다.

식별자 값	용도	비고
0x510 ~ 0x512	차량 상태/조향/구간 문맥 전달	UDP 식별자
0x206	최종 경고 결과 배포	UDP 식별자 (Event + 50ms)
0xE100	긴급 알림 송수신	UDP 식별자 (100ms)

0.3.7 5. 예약 구간 (식별자 정책)

구간	용도	적용
0x500 ~ 0x50F	Ethernet 경계 확장 예약	Ethernet/UDP 식별자
0x513 ~ 0x53F	Backbone/CGW 확장 예약	Ethernet/UDP 식별자
0x700 ~ 0x7FF	Validation/Diagnostic 확장 예약	CAN 11-bit

0.3.8 6. 배정 절차

1. 메시지의 Owner ECU와 도메인을 정한다.
2. CAN 메시지는 0x100 ~ 0x2FF 활성 대역에서 선택하고, Ethernet 메시지는 프로토콜 식별자 정책으로 관리한다.
3. 같은 도메인 안에서 중복 없이 다음 ID를 배정한다.
4. 0303, DBC, 시험 문서까지 같은 ID로 맞춘다.

0.3.9 7. 운영 메모

- 제출 문서 기준에서는 현재 실행 중인 11-bit ID 체계를 사용한다.
- 29-bit 확장 정책은 진단 및 향후 확장 검토 항목으로 live SoT에서 별도로 관리한다.
- 신규 메시지는 기존 활성 대역과 예약 구간 충돌 여부를 먼저 확인한다.

0.4 Master Test Matrix

Document ID: PROJ-00G-MTM **ISO 26262 Reference**: Part 4 / Part 6 (Verification Planning and Qualification) **ASPICE Reference**: SYS.4 / SYS.5 / SWE.4 / SWE.5 / SWE.6 **Version**: 0.2 **Date**: 2026-03-15 **Status**: Draft Baseline **Project Title**: 주행 상황 실시간 경고 시스템 **Subtitle**: 구간 정보 및 긴급차량 접근 기반 앰비언트·클러스터 경보

> 경고: canoe/src/capl 구현이 추가 개발 또는 업데이트되면, 같은 변경 범위의 Master Test Matrix, 05/06/07, oracle/evidence 정의, executable TEST 자산도 동일 기준선으로 즉시 동기화해야 한다. CAPL과 TEST 기준선 불일치는 그대로 추적성 실패로 간주한다.
>
> 참고: 본 문서는 active executable baseline만 유지한다. retired umbrella row 이력은 00g_Master_Test_Matrix_retired.md에서 별도로 관리한다.

0.4.1 1. 목적

본 문서는 현재 프로젝트의 상위 테스트 통제 문서이다.

목적은 다음과 같다.

1. Req -> UT/IT/ST 추적성을 하나의 표에서 통제한다.
2. 안전 항목과 일반 기능 항목을 같은 테스트 표면에서 구분한다.
3. native CANoe Test Unit 구현 전에 상위 검증 기준을 먼저 고정한다.

0.4.2 2. 컬럼 운영 원칙

1. Test Level은 UT / IT / ST만 사용한다.
2. ASIL/QM는 Test Level에 넣지 않고 별도 컬럼으로 관리한다.
3. Safety Goal은 HARA 요약본 기준으로 연결한다.
4. HARA 직접 연결이 아직 없는 항목은 QM 또는 TBD로 시작한다.
5. Status는 Planned -> Ready -> Pass/Fail 순서로만 관리한다.
6. 05_Unit_Test.md의 Simulator 입력/출력 지원 행은 formal requirement ownership이 잠기기 전까지 본 Matrix의 주 표면에 올리지 않는다.
7. UT 승격 판단은 04_SW_Implementation.md 대신 canoe/src/capl actual ownership을 우선 기준으로 사용한다.

0.4.3 3. 01 문서와의 관계

현재 01_Requirements.md는 중요도/긴급도 축을 가진다.

안전 추적성을 더 명확히 하려면 다음 열을 중요도/긴급도 옆에 별도로 두는 것이 좋다.

- HARA ID
- ASIL/QM
- Safety Goal Ref

이 축은 우선순위와 다르므로 같은 열에 섞지 않는다.

0.4.4 4. Master Test Matrix (Draft Baseline)

Req ID	HARA
Req_007, Req_008, Req_009, Req_010	HC-01
Req_017, Req_018, Req_019, Req_020, Req_022, Req_023	HC-03,
Req_049, Req_050, Req_051, Req_052, Req_053	-

Req ID	HARA
Req_057, Req_058, Req_059, Req_060, Req_061, Req_062, Req_063, Req_064, Req_065, Req_075	HC-06
Req_069, Req_070, Req_073	-
Req_054, Req_055, Req_056, Req_093	HC-07
Req_037, Req_038, Req_040, Req_041, Req_042	-
Req_007, Req_008, Req_009, Req_010, Req_013, Req_014, Req_015, Req_016	HC-01
Req_017, Req_018, Req_019, Req_020, Req_022, Req_023, Req_024, Req_096, Req_097	HC-03,
Req_021, Req_024, Req_026, Req_027, Req_028, Req_029, Req_030, Req_031	HC-03
Req_007, Req_008, Req_009, Req_010, Req_013, Req_014, Req_015, Req_016	HC-01
Req_033, Req_034, Req_035	HC-04
Req_018, Req_019, Req_020, Req_036	HC-03
Req_033, Req_034, Req_035	HC-04
Req_018, Req_019, Req_020, Req_036, Req_070, Req_073	HC-03
Req_040, Req_083	HC-02
Req_084	HC-08
Req_043, Req_085	-
Req_086	HC-08
Req_045, Req_046, Req_047, Req_087	-
Req_088	-
Req_089	-
Req_090, Req_092	HC-06
Req_091, Req_092	HC-06
Req_093	HC-07
Req_094, Req_095	HC-08
Req_094, Req_095	HC-08
Req_093	-
Req_093	-
Req_033, Req_034, Req_035	HC-04
Req_018, Req_019, Req_020, Req_070, Req_073, Req_074, Req_088	HC-03
Req_018, Req_019, Req_020, Req_036, Req_073	HC-03
Req_018, Req_019, Req_020, Req_073, Req_088	HC-03
Req_048, Req_074, Req_089	HC-08
Req_049, Req_050, Req_051, Req_052, Req_053	-
Req_096	-
Req_097	-
Req_001, Req_002, Req_003, Req_004, Req_005, Req_006	-
Req_007, Req_008, Req_009, Req_010	HC-01
Req_011, Req_012	HC-02
Req_017, Req_018, Req_019, Req_020	HC-03
Req_017, Req_018, Req_019, Req_020	HC-03
Req_021, Req_024, Req_026, Req_027, Req_028, Req_029, Req_030, Req_031	HC-03
Req_008, Req_009, Req_013, Req_014, Req_015, Req_016, Req_021, Req_026, Req_033, Req_034, Req_035	HC-01,
Req_005, Req_018, Req_019, Req_020, Req_021, Req_026, Req_036	HC-03
Req_022, Req_023, Req_025, Req_032, Req_076, Req_077	HC-04

Req ID	HARA
Req_049, Req_050, Req_051, Req_052, Req_053	-
Req_054, Req_055, Req_056	HC-05
Req_057, Req_058, Req_059, Req_060, Req_061, Req_062, Req_063, Req_064, Req_065, Req_075	HC-06
Req_069	-
Req_073	-
Req_067	-
Req_068	-
Req_074	-
Req_049, Req_050, Req_057, Req_058, Req_059, Req_060, Req_061, Req_062, Req_063, Req_066	HC-06
Req_037	-
Req_038	-
Req_041	HC-02
Req_040	HC-02
Req_039	HC-02
Req_042	-
Req_043	-
Req_044	-
Req_045	-
Req_046	-
Req_046	-
Req_047	-
Req_048	HC-08
Req_078, Req_079	HC-07
Req_081	HC-08
Req_082	HC-08
Req_070, Req_071, Req_072	-
Req_083, Req_084	-
Req_085, Req_086, Req_087	-
Req_088, Req_089	-
Req_090, Req_091, Req_092	-
Req_093	-
Req_094, Req_095	-
Req_067, Req_069, Req_070, Req_073, Req_088, Req_089	-
Req_048, Req_074, Req_075, Req_076, Req_077, Req_080	HC-08
Req_096	-
Req_097	-
Req_001, Req_002, Req_037, Req_044	-
Req_001, Req_002, Req_008, Req_037, Req_038, Req_044	-
Req_001, Req_003, Req_008, Req_039, Req_041	-
Req_004, Req_008	-
Req_007, Req_009, Req_010, Req_015	HC-01
Req_004, Req_010, Req_016	HC-01
Req_007, Req_011, Req_015	HC-02

Req ID	HARA
Req_011, Req_012, Req_015	HC-02
Req_013, Req_014	-
Req_013, Req_014, Req_016	-
Req_017, Req_018, Req_019, Req_020	HC-03
Req_017, Req_018, Req_019, Req_020	HC-03
Req_017, Req_018, Req_019, Req_020	HC-03
Req_017, Req_018, Req_019, Req_020	HC-03
Req_017, Req_018, Req_019, Req_020	HC-03
Req_017, Req_018, Req_019, Req_020	HC-03
Req_017, Req_018, Req_019, Req_020	HC-03
Req_096	-
Req_097	-
Req_022, Req_023, Req_025, Req_032, Req_076, Req_077	HC-04
Req_022, Req_023, Req_025, Req_032, Req_076, Req_077	HC-04
Req_049, Req_050, Req_051, Req_052, Req_053	-
Req_049, Req_050, Req_051, Req_052, Req_053	-
Req_049, Req_050, Req_051, Req_052, Req_053	-
Req_054, Req_055, Req_056	HC-05
Req_054, Req_055, Req_056	HC-05
Req_057, Req_058, Req_059, Req_060, Req_061, Req_062, Req_063, Req_064, Req_065, Req_075	HC-06
Req_057, Req_058, Req_059, Req_060, Req_061, Req_062, Req_063, Req_064, Req_065, Req_075	HC-06
Req_057, Req_058, Req_059, Req_060, Req_061, Req_062, Req_063, Req_064, Req_065, Req_075	HC-06
Req_067, Req_068, Req_069	-
Req_070, Req_073	-
Req_073, Req_074	-
Req_071, Req_072	-
Req_075, Req_080	HC-08
Req_076, Req_077	HC-08
Req_080	HC-08
Req_078, Req_079	HC-07
Req_080, Req_081, Req_082	HC-07,
Req_083, Req_084	-
Req_085, Req_086, Req_087	-
Req_088, Req_089	-
Req_090, Req_091, Req_092	-
Req_093	-
Req_094, Req_095	-
Req_001~Req_032	-
Req_054, Req_055, Req_056, Req_093	HC-05

0.4.5 5. 운영 규칙

1. 본 문서의 Req ID는 01_Requirements.md와 항상 동기화한다.
2. 05/06/07에서 Test ID가 바뀌면 본 문서를 먼저 갱신한다.

3. Pass/Fail 전환 전에 반드시 Oracle, Timing, Evidence가 고정되어 있어야 한다.
4. ASIL/QM는 HARA와 분리된 우선순위 정보가 아니므로 중요도/긴급도와 혼합하지 않는다.
5. 진단 항목은 diagnostic-matrix.md와 함께 관리하되, 핵심 경고 체인은 진단 비의존으로 유지한다.
6. UT_028 ~ UT_069와 같은 stimulus/support asset은 05_Unit_Test.md에는 유지하되, formal requirement ownership이 잠기기 전까지는 본 Matrix의 baseline row로 승격하지 않는다.

0.4.6 6. 다음 단계

1. 01_Requirements.md에 HARA ID, ASIL/QM, Safety Goal Ref 열 추가 검토
2. 05/06/07과 본 Matrix의 Req ID / Status / Evidence 동기화를 유지
3. canoe/src/capl actual ownership 기준으로 direct requirement ownership이 확인된 UT만 후속 승격
4. test-asset-mapping.md와 native-test-asset-naming.md 기준으로 executable asset 연결

S2. 런타임 계약 세트

본 별첨은 V2X, ADAS, CGW, 출력 계층의 ownership 경계와 CAN/Ethernet runtime contract를 보강한다.

원문 기준 경로

- canoe/docs/Kor/contracts/communication-matrix.md
- canoe/docs/Kor/contracts/owner-route.md
- canoe/docs/Kor/contracts/layer-separation-policy.md
- canoe/docs/Kor/contracts/ethernet-interface.md
- canoe/docs/Kor/contracts/multibus-policy.md
- canoe/docs/Kor/contracts/panel-sysvar-contract.md
- canoe/docs/Kor/contracts/ethernet-backbone.md
- appendix/source/contracts/network_architecture_driving_alert_active_baseline.puml
- appendix/source/contracts/signal_flow_integrated_warning_network.puml

네트워크 설계도

본 별첨의 네트워크 설계도는 현재 canoe 실행 구조를 기준으로 재작성하였다. 문서 최신화가 완전히 끝나지 않은 항목은 V2X, ADAS, CGW, TEST_SCN, TEST_BAS의 CAPL 구현과 runtime contract 문서를 함께 대조하여 owner, bus, observer 경계를 확정하였다.

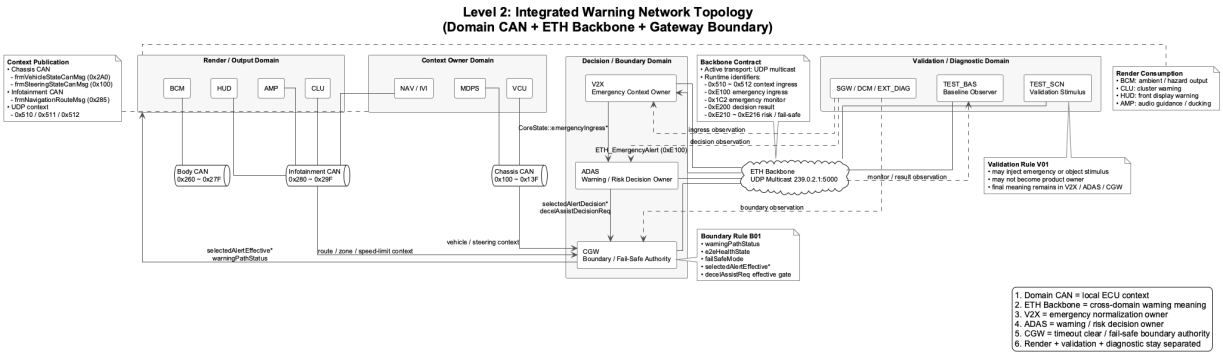

Figure 2: Integrated warning network topology. Domain CAN, ETH Backbone, decision ownership, boundary authority, render/output, validation/diagnostic separation을 한 장에서 정리하였다.

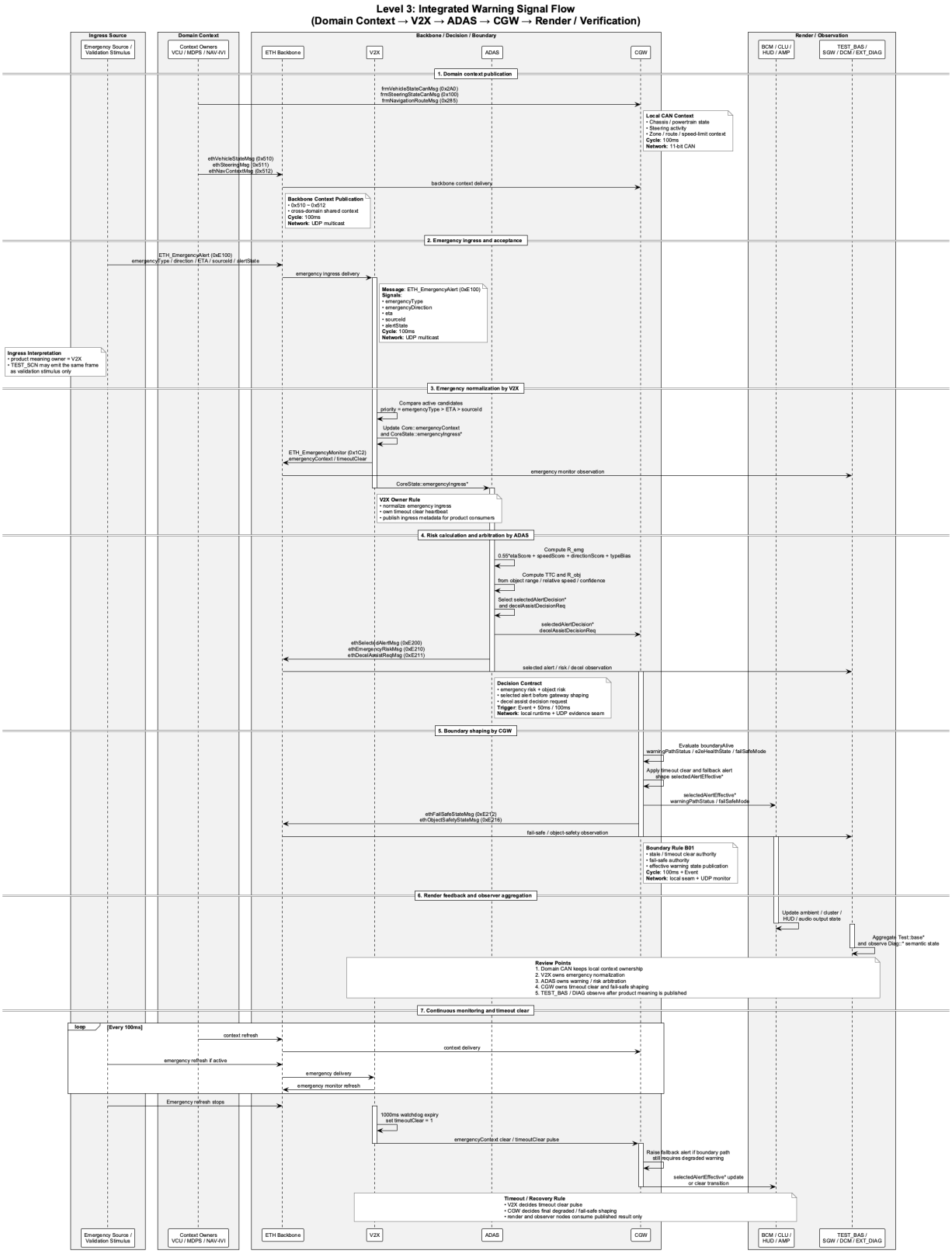

Figure 3: Integrated warning signal flow. Context publication, emergency ingress, V2X normalization, ADAS arbitration, CGW boundary shaping, render/verification, timeout recovery 규칙을 순차적으로 정리하였다.

0.5 Runtime Message Ownership Matrix

원문:

- ../../contracts/communication-matrix.md

동기화 기준:

- 5d83ee7f
- message name, ECU name, DBC 이름, backbone source 값은 canonical technical identifier로 유지합니다.

Important

이 문서는 현재 개발 baseline과 계획 중인 target architecture를 반영합니다. runtime, diagnostic, verification 세부사항 가운데 일부는 아직 구현 중이며 변경될 수 있습니다.

- 생성 시각: 2026-03-12 23:58:00
- 범위: domain CAN과 Ethernet backbone contract에 대한 active runtime ownership
- 규칙: active runtime profile에서 각 message는 명확한 sender 1개를 가져야 합니다.

0.5.1 특수 해석 규칙

- frmEmergencyBroadcastMsg (0x1C0)는 SIL backbone emergency ingress contract입니다.
  - legacy CAN-stub 이름은 retire 상태입니다.
  - active UDP multicast contract는 V2X가 송신하는 ETH_EmergencyAlert (0xE100)입니다.
  - TEST_SCN이 같은 transport contract를 validation ingress stimulus로 emit할 수 있지만, 이 matrix에서는 product sender로 계산하지 않습니다.
  - direction, ETA, source 메타데이터가 필요한 product consumer는 raw V2X::* transport mirror 대신 V2X가 publish한 normalized CoreState ingress seam을 사용해야 합니다.
- ethSelectedAlertMsg는 현재 SIL profile에서 ADAS-owned backbone publication seam으로 유지됩니다.
  - active runtime에서 BCM, IVI, CLU는 direct backbone RX owner가 아닙니다.
  - 이 node들은 CoreState::selectedAlertEffective*를 먼저 소비하고, 짧은 과도 구간에서는 CoreState::selectedAlertDecision*, 그 다음 documented local render/output fallback을 순서대로 사용합니다.
  - ethSelectedAlertMsg는 active SIL baseline의 output/evidence seam으로 해석해야 하며, 해당 consumer node의 direct ingress path로 강제 해석하면 안 됩니다.
- owner, observer, validation 예외를 포함한 계층 분리 기준은 contracts/layer-separation-policy.md를 따릅니다.
- backbone contract row는 source column에 UDP multicast (239.0.2.1:5000)을 사용합니다.
- validation result aggregation은 Test::scenarioResult와 Test::base* sysvar를 사용하므로, 해당 seam에는 active network message를 두지 않습니다.

메시지	ID (hex)	DLC	송신자	계약 원본	신호
frmSteeringStateCanMsg	0x100	1	MDPS	chassis_can.dbc	2
frmWheelSpeedMsg	0x101	4	ESC	chassis_can.dbc	4
frmYawAccelMsg	0x102	4	ESC	chassis_can.dbc	2
frmChassisHealthMsg	0x103	2	ESC	chassis_can.dbc	3
frmWheelPulseMsg	0x104	2	VCU	chassis_can.dbc	2
frmSuspensionStateMsg	0x105	2	ESC	chassis_can.dbc	3
frmTirePressureMsg	0x106	4	TPMS	chassis_can.dbc	4
frmChassisDiagResMsg	0x107	3	ESC	chassis_can.dbc	3
frmRoadFrictionMsg	0x108	1	ESC	chassis_can.dbc	1

메시지	ID (hex)	DLC	송신자	계약 원본	신호
frmPowertrainGatewayMsg	0x109	2	VCU	powertrain_can.dbc	2
frmVehicleModeMsg	0x10A	2	VCU	powertrain_can.dbc	6
frmPowerLimitMsg	0x10B	2	VCU	powertrain_can.dbc	2
frmCruiseStateMsg	0x10C	2	SCC	powertrain_can.dbc	4
frmPowertrainHealthMsg	0x10D	2	VCU	powertrain_can.dbc	3
frmPtDiagResMsg	0x10E	3	VCU	powertrain_can.dbc	3
frmEnergyFlowStateMsg	0x10F	2	VCU	powertrain_can.dbc	3
frmPowertrainCtrlAuthMsg	0x110	1	VCU	powertrain_can.dbc	3
ethFailSafeStateMsg	0xE212	2	CGW	UDP multicast (239.0.2.1:5000)	5
frmBrakeStatusMsg	0x120	2	ESC	chassis_can.dbc	5
frmAccelStatusMsg	0x121	2	VCU	chassis_can.dbc	2
frmSteeringTorqueMsg	0x122	2	MDPS	chassis_can.dbc	2
frmEpsStateMsg	0x123	2	MDPS	chassis_can.dbc	4
frmAbsStateMsg	0x124	2	ABS	chassis_can.dbc	3
frmEscStateMsg	0x125	2	ESC	chassis_can.dbc	3
frmTcsStateMsg	0x126	2	VCU	chassis_can.dbc	3
frmBrakeTempMsg	0x127	2	ESC	chassis_can.dbc	2
frmSteeringAngleMsg	0x128	2	SAS	chassis_can.dbc	1
frmBrakeWearMsg	0x129	1	ESC	chassis_can.dbc	1
frmEngineSpeedTempMsg	0x12A	4	EMS	powertrain_can.dbc	3
frmEpbStateMsg	0x12A	2	EPB	chassis_can.dbc	4
frmFuelBatteryStateMsg	0x12B	3	EMS	powertrain_can.dbc	4
frmVsmStateMsg	0x12B	2	VSM	chassis_can.dbc	3
frmEhbStateMsg	0x12C	2	EHB	chassis_can.dbc	2
frmThrottleStateMsg	0x12C	2	EMS	powertrain_can.dbc	2
frmEcsStateMsg	0x12D	2	ECS	chassis_can.dbc	3
frmTransmissionTempMsg	0x12D	2	TCU	powertrain_can.dbc	2
frmCdcStateMsg	0x12E	2	CDC	chassis_can.dbc	2
frmEngineTorqueMsg	0x12E	2	EMS	powertrain_can.dbc	1
frmAirSuspensionStateMsg	0x12F	3	ASM	chassis_can.dbc	3
frmEngineLoadMsg	0x12F	1	EMS	powertrain_can.dbc	1
frmRwsStateMsg	0x130	3	RWS	chassis_can.dbc	3
frmTransShiftStateMsg	0x130	2	TCU	powertrain_can.dbc	4
frmThermalMgmtStateMsg	0x131	2	EMS	powertrain_can.dbc	3
frmObcStateMsg	0x132	2	OBC	powertrain_can.dbc	4
frmDcdcStateMsg	0x133	3	DCDC	powertrain_can.dbc	4
frmMcuStateMsg	0x134	4	MCU	powertrain_can.dbc	4
frmInverterStateMsg	0x135	3	INVERTER	powertrain_can.dbc	4
frm4wdStateMsg	0x136	2	_4WD	powertrain_can.dbc	3
frmBatBmsStateMsg	0x137	3	BAT_BMS	powertrain_can.dbc	3
frmFpcmStateMsg	0x138	3	FPCM	powertrain_can.dbc	4
frmLvrStateMsg	0x139	2	LVR	powertrain_can.dbc	6
frmIsgStateMsg	0x13A	2	ISG	powertrain_can.dbc	4

메시지	ID (hex)	DLC	송신자	계약 원본	신호
frmEopStateMsg	0x13B	4	EOP	powertrain_can.dbc	4
frmEwpStateMsg	0x13C	3	EWP	powertrain_can.dbc	4
frmChargePortCtrlStateMsg	0x13D	3	CPC	powertrain_can.dbc	7
ETH_EmergencyAlert	0xE100	4	V2X	UDP multicast (239.0.2.1:5000)	5
frmAdasChassisStatusMsg	0x1C1	2	ADAS	adas_can.dbc	2
ETH_EmergencyMonitor	0x1C2	2	V2X	UDP multicast (239.0.2.1:5000)	2
ethEmergencyRiskMsg	0xE210	5	ADAS	UDP multicast (239.0.2.1:5000)	6
ethDecelAssistReqMsg	0xE211	4	ADAS	UDP multicast (239.0.2.1:5000)	6
ethObjectRiskInputMsg	0xE213	8	TEST_SCN	UDP multicast (239.0.2.1:5000)	8
ethObjectRiskStateMsg	0xE214	6	ADAS	UDP multicast (239.0.2.1:5000)	5
ethObjectScenarioAlertMsg	0xE215	4	ADAS	UDP multicast (239.0.2.1:5000)	6
ethObjectSafetyStateMsg	0xE216	4	CGW	UDP multicast (239.0.2.1:5000)	5
frmLdwsLkasStateMsg	0x1C8	4	LDWS_LKAS	adas_can.dbc	7
frmFcaStateMsg	0x1C9	4	FCA	adas_can.dbc	7
frmBcwStateMsg	0x1CA	3	BCW	adas_can.dbc	6
frmLcaStateMsg	0x1CB	3	LCA	adas_can.dbc	6
frmSpasStateMsg	0x1CC	4	SPAS	adas_can.dbc	5
frmRspaStateMsg	0x1CD	4	RSPA	adas_can.dbc	5
frmAvmStateMsg	0x1CE	4	AVM	adas_can.dbc	5
frmFcamStateMsg	0x1CF	4	FCAM	adas_can.dbc	5
frmFradarStateMsg	0x1D0	4	FRADAR	adas_can.dbc	5
frmSrrFlStateMsg	0x1D1	4	SRR_FL	adas_can.dbc	5
frmSrrFrStateMsg	0x1D2	4	SRR_FR	adas_can.dbc	5
frmParkUltrasonicStateMsg	0x1D3	4	PUS	adas_can.dbc	5
frmDmsStateMsg	0x1D4	4	DMS	adas_can.dbc	5
frmOmsStateMsg	0x1D5	4	OMS	adas_can.dbc	6
frmSrrRlStateMsg	0x1D6	4	SRR_RL	adas_can.dbc	5
frmSrrRrStateMsg	0x1D7	4	SRR_RR	adas_can.dbc	5
frmAebDomainStateMsg	0x1D8	4	AEB	adas_can.dbc	7
frmParkMasterStateMsg	0x1D9	4	PKM	adas_can.dbc	8
frmRoadPreviewCameraStateMsg	0x1DA	4	RPC	adas_can.dbc	6
frmRearRadarMasterStateMsg	0x1DB	4	RRM	adas_can.dbc	7
frmSurroundParkMasterStateMsg	0x1DC	4	SPM	adas_can.dbc	7
frmHighwayPilotStateMsg	0x1DD	4	HWP	adas_can.dbc	9
frmLidarStateMsg	0x1DE	4	LDR	adas_can.dbc	5
frmTrailerCtrlStateMsg	0x1DF	4	TRM	adas_can.dbc	6
frmAdasDiagReqMsg	0x1E0	3	TEST_SCN	adas_can.dbc	3
frmAdasDiagResMsg	0x1E1	3	ADAS	adas_can.dbc	3
frmDmsDiagReqMsg	0x1E2	3	TEST_SCN	adas_can.dbc	3
frmDmsDiagResMsg	0x1E3	3	DMS	adas_can.dbc	3
frmOmsDiagReqMsg	0x1E4	3	TEST_SCN	adas_can.dbc	3
frmOmsDiagResMsg	0x1E5	3	OMS	adas_can.dbc	3
ethSelectedAlertMsg	0xE200	2	ADAS	UDP multicast (239.0.2.1:5000)	4

메시지	ID (hex)	DLC	송신자	계약 원본	신호
frmAmbientControlMsg	0x260	1	BCM	body_can.dbc	3
frmHazardControlMsg	0x261	1	BCM	body_can.dbc	3
frmWindowControlMsg	0x262	1	BCM	body_can.dbc	3
frmDoorStateMsg	0x264	2	BCM	body_can.dbc	5
frmLampControlMsg	0x265	1	BCM	body_can.dbc	5
frmWiperStateMsg	0x266	1	WIP	body_can.dbc	3
frmSeatBeltStateMsg	0x267	1	BCM	body_can.dbc	5
frmCabinAirStateMsg	0x268	2	BCM	body_can.dbc	2
frmBodyHealthMsg	0x269	2	BCM	body_can.dbc	3
frmHvacStateMsg	0x26A	2	DATC	body_can.dbc	4
frmHvacActuatorMsg	0x26B	2	DATC	body_can.dbc	5
frmMirrorStateMsg	0x26C	1	MIR	body_can.dbc	4
frmSeatStateMsg	0x26D	2	BCM	body_can.dbc	2
frmSeatControlMsg	0x26E	2	BCM	body_can.dbc	4
frmDoorControlMsg	0x26F	1	BCM	body_can.dbc	3
frmInteriorLightMsg	0x270	1	BCM	body_can.dbc	2
frmRainLightAutoMsg	0x271	1	BCM	body_can.dbc	3
frmBcmDiagReqMsg	0x272	3	TEST_SCN	body_can.dbc	3
frmBcmDiagResMsg	0x273	3	BCM	body_can.dbc	3
frmImmobilizerStateMsg	0x274	1	SMK	body_can.dbc	3
frmAlarmStateMsg	0x275	1	BSEC	body_can.dbc	3
frmBodyGatewayStateMsg	0x276	2	BCM	body_can.dbc	2
frmBodyComfortStateMsg	0x277	2	BCM	body_can.dbc	3
frmAflsStateMsg	0x278	2	AFLS	body_can.dbc	4
frmDoorFlStateMsg	0x279	2	DOOR_FL	body_can.dbc	3
frmDoorFrStateMsg	0x27A	2	DOOR_FR	body_can.dbc	3
frmSeatDrvStateMsg	0x27B	2	SEAT_DRV	body_can.dbc	3
frmSeatPassStateMsg	0x27C	2	SEAT_PASS	body_can.dbc	3
frmDoorRlStateMsg	0x27D	2	DOOR_RL	body_can.dbc	3
frmDoorRrStateMsg	0x27E	2	DOOR_RR	body_can.dbc	3
frmTailgateStateMsg	0x27F	2	TGM	body_can.dbc	4
frmClusterWarningMsg	0x280	1	IVI	infotainment_can.dbc	1
frmRearClimateStateMsg	0x280	2	RATC	body_can.dbc	4
frmClusterBaseStateMsg	0x281	2	IVI	infotainment_can.dbc	3
frmSunroofStateMsg	0x281	2	SRF	body_can.dbc	4
frmHeadlampLevelStateMsg	0x282	2	HLM	body_can.dbc	4
frmNaviGuideStateMsg	0x282	1	NAV	infotainment_can.dbc	2
frmCabinSensingStateMsg	0x283	2	CSM	body_can.dbc	4
frmMediaStateMsg	0x283	2	IVI	infotainment_can.dbc	5
frmAhlsStateMsg	0x284	2	AHLS	body_can.dbc	3
frmCallStateMsg	0x284	2	IVI	infotainment_can.dbc	5
frmAutoDoorCtrlStateMsg	0x285	2	ADM	body_can.dbc	3
frmNavigationRouteMsg	0x285	3	NAV	infotainment_can.dbc	5

메시지	ID (hex)	DLC	송신자	계약 원본	신호
frmClusterThemeMsg	0x286	1	IVI	infotainment_can.dbc	2
frmPowerTailgateCtrlStateMsg	0x286	2	PTG	body_can.dbc	3
frmBiometricAuthStateMsg	0x287	2	BIO	body_can.dbc	3
frmHmiPopupStateMsg	0x287	1	IVI	infotainment_can.dbc	3
frmAcuStateMsg	0x288	2	ACU	body_can.dbc	5
frmInfotainmentHealthMsg	0x288	2	IVI	infotainment_can.dbc	3
frmAudioFocusMsg	0x289	1	IVI	infotainment_can.dbc	3
frmOdsStateMsg	0x289	2	ODS	body_can.dbc	5
frmMassageSeatCtrlStateMsg	0x28A	3	MSC	body_can.dbc	6
frmVoiceAssistStateMsg	0x28A	1	VCS	infotainment_can.dbc	3
frmDatcDiagReqMsg	0x28B	3	TEST_SCN	body_can.dbc	3
frmMapRenderStateMsg	0x28B	2	NAV	infotainment_can.dbc	3
frmDatcDiagResMsg	0x28C	3	DATC	body_can.dbc	3
frmRouteAlertMsg	0x28C	2	NAV	infotainment_can.dbc	4
frmTrafficEventMsg	0x28D	3	NAV	infotainment_can.dbc	3
frmPhoneProjectionMsg	0x28E	1	IVI	infotainment_can.dbc	3
frmClusterNotifMsg	0x28F	2	IVI	infotainment_can.dbc	2
frmIviDiagResMsg	0x290	3	IVI	infotainment_can.dbc	3
frmMediaMetaMsg	0x291	2	IVI	infotainment_can.dbc	4
frmSpeechTtsStateMsg	0x292	2	VCS	infotainment_can.dbc	4
frmConnectivityStateMsg	0x293	2	IVI	infotainment_can.dbc	4
frmIviHealthDetailMsg	0x294	2	IVI	infotainment_can.dbc	2
frmClusterSyncStateMsg	0x295	2	IVI	infotainment_can.dbc	3
frmTmuServiceStateMsg	0x296	2	TMU	infotainment_can.dbc	6
frmHudStateMsg	0x297	4	HUD	infotainment_can.dbc	5
frmAmpStateMsg	0x298	3	AMP	infotainment_can.dbc	7
frmOtaMasterStateMsg	0x299	4	OTA	infotainment_can.dbc	9
frmDigitalKeyStateMsg	0x29A	3	DKEY	infotainment_can.dbc	7
frmRseStateMsg	0x29B	3	RSE	infotainment_can.dbc	7
frmNavModuleStateMsg	0x29C	3	NAV	infotainment_can.dbc	6
frmPgsStateMsg	0x29D	2	PGS	infotainment_can.dbc	4
frmPhoneAsKeyStateMsg	0x29E	2	PAK	infotainment_can.dbc	5
frmCarpayCtrlStateMsg	0x29F	2	CPAY	infotainment_can.dbc	5
frmTmuDiagReqMsg	0x2A0	3	TEST_SCN	infotainment_can.dbc	3
frmVehicleStateCanMsg	0x2A0	2	VCU	chassis_can.dbc	3
frmSteeringCanMsg	0x2A1	1	TEST_SCN	chassis_can.dbc	2
frmTmuDiagResMsg	0x2A1	3	TMU	infotainment_can.dbc	3
frmPedalInputCanMsg	0x2A2	2	TEST_SCN	chassis_can.dbc	2
frmNavContextCanMsg	0x2A3	3	TEST_SCN	infotainment_can.dbc	5
frmChassisDiagReqMsg	0x2A4	3	TEST_SCN	chassis_can.dbc	3
frmIviDiagReqMsg	0x2A7	3	TEST_SCN	infotainment_can.dbc	3
frmCluDiagReqMsg	0x2A8	3	TEST_SCN	infotainment_can.dbc	3
frmIgnitionEngineMsg	0x2A8	1	TEST_SCN	powertrain_can.dbc	3

메시지	ID (hex)	DLC	송신자	계약 원본	신호
frmCluDiagResMsg	0x2A9	3	CLU	infotainment_can.dbc	3
frmGearStateMsg	0x2A9	1	TEST_SCN	powertrain_can.dbc	3
frmHudDiagReqMsg	0x2AA	3	TEST_SCN	infotainment_can.dbc	3
frmPtDiagReqMsg	0x2AA	3	TEST_SCN	powertrain_can.dbc	3
frmHudDiagResMsg	0x2AB	3	HUD	infotainment_can.dbc	3
frmSccDiagReqMsg	0x2AB	3	TEST_SCN	powertrain_can.dbc	3
frmAmpDiagReqMsg	0x2AC	3	TEST_SCN	infotainment_can.dbc	3
frmSccDiagResMsg	0x2AC	3	SCC	powertrain_can.dbc	3
frmAmpDiagResMsg	0x2AD	3	AMP	infotainment_can.dbc	3
frmAbsDiagReqMsg	0x2BC	3	TEST_SCN	chassis_can.dbc	3
frmAbsDiagResMsg	0x2BD	3	ABS	chassis_can.dbc	3
frmEpbDiagReqMsg	0x2BE	3	TEST_SCN	chassis_can.dbc	3
frmEpbDiagResMsg	0x2BF	3	EPB	chassis_can.dbc	3
frmTpmsDiagReqMsg	0x2C0	3	TEST_SCN	chassis_can.dbc	3
frmTpmsDiagResMsg	0x2C1	3	TPMS	chassis_can.dbc	3
frmSasDiagReqMsg	0x2C2	3	TEST_SCN	chassis_can.dbc	3
frmSasDiagResMsg	0x2C3	3	SAS	chassis_can.dbc	3
frmVsmDiagReqMsg	0x2C4	3	TEST_SCN	chassis_can.dbc	3
frmVsmDiagResMsg	0x2C5	3	VSM	chassis_can.dbc	3
frmEhbDiagReqMsg	0x2C6	3	TEST_SCN	chassis_can.dbc	3
frmEhbDiagResMsg	0x2C7	3	EHB	chassis_can.dbc	3
frmEcsDiagReqMsg	0x2C8	3	TEST_SCN	chassis_can.dbc	3
frmEcsDiagResMsg	0x2C9	3	ECS	chassis_can.dbc	3
frmCdcDiagReqMsg	0x2CA	3	TEST_SCN	chassis_can.dbc	3
frmCdcDiagResMsg	0x2CB	3	CDC	chassis_can.dbc	3
frmAcuDiagReqMsg	0x2CC	3	TEST_SCN	chassis_can.dbc	3
frmAcuDiagResMsg	0x2CD	3	ACU	chassis_can.dbc	3
frmOdsDiagReqMsg	0x2CE	3	TEST_SCN	chassis_can.dbc	3
frmOdsDiagResMsg	0x2CF	3	ODS	chassis_can.dbc	3
frmSmkDiagReqMsg	0x2D0	3	TEST_SCN	body_can.dbc	3
frmSmkDiagResMsg	0x2D1	3	SMK	body_can.dbc	3
frmAflsDiagReqMsg	0x2D2	3	TEST_SCN	body_can.dbc	3
frmAflsDiagResMsg	0x2D3	3	AFLS	body_can.dbc	3
frmWipDiagReqMsg	0x2D4	3	TEST_SCN	body_can.dbc	3
frmWipDiagResMsg	0x2D5	3	WIP	body_can.dbc	3
ethVehicleStateMsg	0x510	6	VCU	UDP multicast (239.0.2.1:5000)	6
ethSteeringMsg	0x511	4	MDPS	UDP multicast (239.0.2.1:5000)	4
ethNavContextMsg	0x512	6	IVI	UDP multicast (239.0.2.1:5000)	5

0.6 Owner / Bus / Timeout / Route Contract

원문:

- ../../contracts/owner-route.md

동기화 기준:

- 5d83ee7f
- seam 이름, owner, bus 이름은 canonical technical string으로 유지합니다.

Important

이 문서는 현재 개발 baseline과 계획 중인 target architecture를 반영합니다. runtime, diagnostic, verification 세부사항 가운데 일부는 아직 구현 중이며 변경될 수 있습니다.

0.6.1 목적

이 문서는 주요 runtime seam마다 누가 owner인지, 어떤 bus가 적용되는지, 그리고 timeout authority와 route authority가 누구인지 정의합니다.

0.6.2 계약 차원

차원	의미
Owner	해당 seam의 business meaning을 소유하는 runtime authority
Bus	active baseline에서 사용하는 primary transport 또는 observation seam
Timeout authority	stale-state, clear, fail-safe 동작을 결정하는 runtime authority
Route authority	cross-domain delivery와 forwarding을 제어하는 runtime authority

0.6.3 규칙

- 하나의 logical seam은 하나의 explicit owner를 가져야 합니다
- cross-domain forwarding이 개입되면 owner와 route authority는 달라질 수 있습니다
- timeout 및 clear 동작은 transport만으로 추정하지 말고 명시적으로 정의해야 합니다
- 계층 분리와 observer 분리 원칙은 contracts/layer-separation-policy.md를 따릅니다
- foreign-domain CAN visibility는 contracts/multibus-policy.md를 따릅니다
- 상세 frame-level ownership은 contracts/communication-matrix.md에서 관리합니다

0.6.4 현재 seam 표

Seam	Owner	Primary bus
Navigation context	IVI	Infotainment CAN
Emergency context	V2X	Ethernet backbone + normalized Core/CoreState seam
Arbitration decision	ADAS	local runtime + CoreState::selectedAlertDecision* sean
Arbitration effective result	CGW	CoreState::selectedAlertEffective* seam + published o
Decel decision	ADAS	local runtime + @Core::decelAssistDecisionReq + Cor
Decel effective result	CGW	@Core::decelAssistReq + CoreState::decelGateReason
Render warning state	IVI, CLU, HUD, AMP	local render output + CoreState::selectedAlertEffectiv
Body ambient / hazard output	BCM	body output + CoreState::selectedAlertEffective* cons
Boundary health	CGW	Ethernet backbone
Scenario result	TEST_SCN	test harness seam

Seam	Owner	Primary bus
Baseline result	TEST_BAS	SysVar-only seam

0.6.5 사용 규칙

seam-level authority 판단은 이 문서를 기준으로 합니다. 해당 seam이 transport, decision, boundary, render, observer, diagnostic semantic 가운데 무엇인지 먼저 판단할 때는 contracts/layer-separation-policy.md를 먼저 봅니다. frame-level ownership을 확인할 때는 contracts/communication-matrix.md를 사용합니다.

0.6.6 개발 메모

위 seam 표는 active baseline의 현재 계획 계약입니다. 개발 중 seam의 transport 또는 ownership이 바뀌면, 그 변경을 공식으로 취급하기 전에 먼저 이 문서를 갱신해야 합니다.

0.7 Runtime Layer Separation Policy

원문:

- ../../contracts/layer-separation-policy.md

동기화 기준:

- canonical english source
- seam 이름, ECU 이름, message name, SysVar name은 canonical technical string으로 유지합니다.

Important

이 문서는 현재 개발 baseline과 계획 중인 target architecture를 반영합니다. 일부 runtime, diagnostic, verification 세부사항은 아직 구현 중이며 변경될 수 있습니다.

0.7.1 목적

이 문서는 active CANoe SIL baseline에서 transport / decision / boundary / render / observer / diagnostic 계층을 어떻게 분리할지 정의합니다.

이 문서가 답하는 질문은 다음과 같습니다.

- 어떤 node가 해당 seam의 business meaning owner인가
- 어떤 node가 그 seam을 transport 또는 route만 할 수 있는가
- 어떤 node가 observer 또는 validation 역할만 해야 하는가
- 어떤 seam이 operational, render-facing, diagnostic, validation-only인지

0.7.2 외부 기준

이 분리 원칙은 다음 공개 기준을 바탕으로 정리했습니다.

- Vector CAPL Callback Interface in CANoe
  - tester side와 ECU simulation side를 구분하고, target/channel을 명시적으로 다룹니다.
- Vector Ethernet_E: DoIP
  - diagnostic tester, diagnostic gateway, DoIP node를 구분합니다.
- AUTOSAR Diagnostic Communication Manager
  - external diagnostic tool과 onboard DCM을 구분하고, diagnostic state를 network-independent하게 관리합니다.
- AUTOSAR PDU Router
  - router는 I-PDU를 forward/routing하지만 payload 의미를 다시 정의하지 않습니다.
- ISO 13400-2
  - client tester, vehicle gateway, sub-component routing 구조를 정의합니다.

0.7.3 계층 모델

active baseline은 아래 여섯 계층으로 나눕니다.

Layer	의미	전형적인 owner
Transport ingress	raw network context 수신 및 freshness 정규화	ingress owner
Functional decision	warning, risk, assist 의미 결정	decision owner
Boundary / override	fail-safe, route, freshness 정책 적용	gateway / boundary owner
Render / output	body, display, cluster, audio로 결과 반영	render owner
Observer / validation	이미 publish된 seam을 evidence/verdict 용도로 관측	validation harness
Diagnostic semantic	diagnostic state, session, security, response 의미 제공	diagnostic owner

0.7.4 분리 규칙

1. logical seam마다 business owner는 하나

- 하나의 logical seam은 하나의 business owner만 가집니다.
- compatibility mirror, trace seam, validation observer는 ownership을 바꾸지 않습니다.
- validation injector가 존재해도 product owner가 바뀌면 안 됩니다.

2. transport와 decision ownership은 다릅니다

- transport를 수신한다고 decision owner가 되는 것은 아닙니다.
- gateway/router는 seam을 forward할 수 있지만 seam의 business meaning을 다시 정의하면 안 됩니다.

3. boundary policy는 중앙 authority가 가져야 합니다

- stale, timeout-clear, degraded, fail-safe는 하나의 boundary authority가 정의해야 합니다.
- downstream node는 그 상태를 소비해야 하며, 두 번째 freshness policy를 임의로 만들면 안 됩니다.

4. observer는 operational ownership에 개입하지 않습니다

- observer node는 published seam을 구독할 수 있습니다.
- 하지만 alternate product owner가 되면 안 됩니다.
- validation aggregation은 operational owner가 먼저 publish한 뒤에만 허용합니다.
- active validation lifecycle 밖에서는 observer seam이 reset-safe idle 상태를 유지해야 합니다.
- idle transport traffic이 validation observer reset 값을 덮어쓰면 안 됩니다.
- observer freshness나 receive timestamp는 실제 관측 transport 또는 owner-published seam 기준이어야 하며, 하네스가 synthetic하게 추정해서는 안 됩니다.

5. diagnostic semantic과 request/response route는 분리합니다

- compact semantic seam은 둘 수 있습니다.
- 하지만 그 semantic seam을 actual network request/response route의 대체물로 취급하면 안 됩니다.

6. validation-only stimulus는 명시적으로 유지합니다

- TEST_SCN은 SIL에서 validation-only transport stimulus를 넣을 수 있습니다.
- 이 예외는 product owner를 테스트하기 위한 것이며, ownership 재할당을 의미하지 않습니다.
- TEST_SCN은 시험 verdict 상태를 계산할 수 있지만, 그 verdict는 owner가 publish한 seam과 시험 출력 기준으로만 도출해야 합니다.
- product owner가 값을 publish하지 못할 때 TEST_SCN이 fallback semantic owner가 되어서는 안 됩니다.
- TEST_BAS는 observer 결과를 집계할 수 있지만, 관측 seam의 alternate product meaning을 합성해서는 안 됩니다.
- 하네스는 scenario timing을 오케스트레이션할 수 있지만, product owner가 이미 clear한 semantic state를 하네스가 계속 살려 두어서는 안 됩니다.

0.7.5 현재 프로젝트 적용

Emergency / V2X path

Seam	Product owner
ETH_EmergencyAlert	V2X
Core::emergencyContext	V2X
CoreState::emergencyIngressDirection / emergencyIngressEtaSec / emergencyIngressSourceId	V2X
V2X::ingressHeartbeat	V2X
ETH_EmergencyMonitor	V2X
CoreState::selectedAlertDecisionLevel / selectedAlertDecisionType	ADAS
CoreState::selectedAlertEffectiveLevel / selectedAlertEffectiveType / selectedAlertGateReason	CGW
ethSelectedAlertMsg / selected alert result	ADAS transport p
@Core::decelAssistDecisionReq	ADAS
CoreState::driverReleaseReason	ADAS
@Core::decelAssistReq / CoreState::decelGateReason	CGW
ethFailSafeStateMsg / warningPathStatus	CGW

Render / output path

Output meaning	Product owner	Required input
visual warning level/type	IVI, HUD, CLU render side	CoreState::selectedAlertEffective* + do
ambient / body warning actuation	BCM	CGW effective selected-alert state + C
audio focus / ducking / volume guidance	AMP, IVI, VCS render side	CGW effective selected-alert state + u

규칙:

- render node는 owner-published result seam을 소비합니다.
- render consumer는 effective -> decision -> compatibility fallback 순서로 읽는 것을 기본으로 합니다.
- render node가 emergency ingress owner가 되면 안 됩니다.
- render / decision node는 raw V2X::* transport mirror보다 CoreState normalized ingress metadata를 우선 소비해야 합니다.
- raw ingress 직접 참조는 documented compatibility path일 때만 허용합니다.
- TEST_SCN이 직접 emergency transport frame을 주입하는 동안에는 Test::compat* dispatch compatibility 입력을 비워 둡니다. dispatch compatibility stimulus는 dispatch-only 검증 경로에서만 사용하며 V2X::* owner mirror에 직접 쓰면 안 됩니다.

Diagnostic path

Seam	Owner	역할
network request / response route	requester + gateway + ECU server path	실제 transport/addressing fl
Diag::* semantic seam	SGW + DCM	diagnostic status/result 의미
EXT_DIAG	logical external requester / observer surface	현재 baseline에서는 Diag::*만

0.7.6 현재 backbone producer boundary 규칙

active baseline은 backbone seam에서 producer identity를 유지해야 합니다.

Backbone seam	Accepted producer
ETH_EmergencyAlert (current SIL validation ingress)	TEST_SCN
ethObjectRiskInputMsg	TEST_SCN
ETH_EmergencyMonitor	V2X
ethSelectedAlertMsg	ADAS
ethDecelAssistReqMsg	ADAS
ethObjectRiskStateMsg	ADAS
ethObjectScenarioAlertMsg	ADAS
ethFailSafeStateMsg	CGW
ethObjectSafetyStateMsg	CGW
ethValidationOverride	TEST_SCN

의미:

- backbone consumer는 source ownership을 확인한 뒤에만 authoritative frame으로 취급해야 합니다.
- self-originated transport는 strict self-source check로 배제해야 합니다.
- 예상과 다른 external source는 SIL continuity를 위해 trace 후 수용할 수 있지만, 그것이 second logical owner가 되어서는 안 됩니다.
- message ID만 같다고 해서 같은 authority로 보면 안 됩니다.

0.7.7 읽는 순서

1. contracts/layer-separation-policy.md
2. contracts/owner-route.md
3. contracts/ethernet-interface.md
4. contracts/communication-matrix.md
5. contracts/multibus-policy.md

0.7.8 갱신 규칙

seam 역할이 바뀌면 다음 순서로 갱신합니다.

1. contracts/layer-separation-policy.md
2. contracts/owner-route.md
3. contracts/ethernet-interface.md
4. contracts/communication-matrix.md
5. contracts/multibus-policy.md
6. runtime CAPL / verification asset

0.8 ETH Interface Contract

원문:

- ../../contracts/ethernet-interface.md

동기화 기준:

- 5d83ee7f
- seam 이름, owner, consumer 이름, transport 값은 canonical technical string으로 유지합니다.

Important

이 문서는 active CANoe SIL profile에서 실제로 사용하는 Ethernet contract를 정의합니다. bit-level packing 문서가 아니라, active seam과 owner/consumer/fallback 규칙을 정리하는 운영 계약입니다.

0.8.1 1. 목적

이 문서는 active CANoe SIL profile에 대한 operational Ethernet contract를 정의합니다.

이 문서가 답하는 질문은 아래와 같습니다.

- 어떤 Ethernet seam이 active 상태인가
- 각 seam의 logical owner는 누구인가
- 누가 그 seam을 primary consumer로 읽는가
- 어떤 fallback path가 허용되는가
- 어떤 path가 validation-only seam인가

bit field, ID, signal packing 자체는 ethernet-backbone.md를 기준으로 읽습니다. owner, observer, validation, render 계층 분리는 layer-separation-policy.md를 기준으로 읽습니다.

0.8.2 2. Active transport baseline

현재 active backbone transport는 아래 하나로 고정합니다.

- UDP multicast 239.0.2.1:5000

이 transport가 현재 CANoe SIL profile의 inter-domain backbone seam입니다. retired CAN stub seam을 primary architecture contract로 취급하면 안 됩니다.

현재 SIL validation baseline에서는 CANoe multicast runtime stack 동작에 따라 sender identity가 다르게 보일 수 있습니다. 그래서 ingress owner는 아래 원칙을 따릅니다.

- strict self-source check로 self-originated transport를 먼저 배제한다
- 문서화된 validation sender가 보이면 그 값을 우선 사용한다
- 예상과 다른 external source는 trace를 남긴 뒤 external ingress로 수용한다

0.8.3 3. Contract seam 분류

3.1 Input-context seam 이 seam은 raw context를 shared runtime으로 들여오는 입력 경로입니다.

Seam	Logical owner	Primary consumer	의도
ethVehicleStateMsg	VCU	CGW, IBOX	vehicle speed 및 drive state contex
ethSteeringMsg	MDPS	CGW	steering activity context
ethNavContextMsg	IVI	CGW, IBOX	road zone, direction, speed-limit c
ETH_EmergencyAlert	V2X	emergency-context runtime path	emergency ingress context
ethObjectRiskInputMsg	TEST_SCN	ADAS	validation-only object-risk stimulu

3.2 Decision-output seam 이 seam은 arbitration 이후의 selected runtime meaning을 publish 하는 출력 경로입니다.

Seam	Logical owner	Primary consum
ethSelectedAlertMsg	ADAS가 active selected-alert state를 relay하는 transport seam	BCM, IVI, downs
ethEmergencyRiskMsg	ADAS	ADAS-side cons
ethDecelAssistReqMsg	ADAS	ESC, TEST_SCN
ethObjectRiskStateMsg	ADAS	ADAS-side cons
ethObjectScenarioAlertMsg	ADAS	BCM, IVI, TEST_

3.3 Boundary-health seam 이 seam은 health, degradation, observability 상태를 노출합니다.

Seam	Logical owner	Primary consumer	의도
ethFailSafeStateMsg	CGW	ADAS, TEST_SCN, selected safety observer	path health 및 fail-safe
ETH_EmergencyMonitor	V2X	TEST_SCN, trace observer	emergency transport m
ethObjectSafetyStateMsg	CGW	ADAS, TEST_SCN, selected observer	object-path health 및

0.8.4 4. 핵심 규칙

4.1 Business meaning과 transport는 분리한다

- warning의 business meaning을 임시 stub 또는 mirror path에 직접 묶으면 안 됩니다.
- ETH_EmergencyAlert가 현재 active emergency ingress seam입니다.
- Core::* mirror는 SIL compatibility를 도울 수 있지만, published Ethernet contract를 대체하지 않습니다.

4.2 각 seam에는 logical owner를 하나만 둔다

- 각 active Ethernet seam은 current runtime에서 하나의 명확한 logical owner만 가져야 합니다.
- compatibility mirror나 observer가 추가되더라도 ownership은 바뀌지 않습니다.

4.3 Fallback은 문서화된 경우에만 허용한다

- 일부 downstream consumer는 fresh backbone result seam이 없을 때 Core::* 또는 normalized CoreState::* mirror로 fallback할 수 있습니다.
- 이 fallback은 documented compatibility path로만 취급합니다.
- fallback을 primary interface contract처럼 설명하면 안 됩니다.
- ADAS와 render node가 direction, ETA, source 메타데이터가 필요할 때는 V2X owner가 publish한 normalized CoreState::* ingress seam을 우선 사용합니다.

4.4 Validation-only seam은 명시적으로 남긴다

- TEST_SCN은 SIL validation을 위해 ethObjectRiskInputMsg 같은 Ethernet seam을 생성할 수 있습니다.
- 긴급차 ingress path 자체를 시험할 때는 TEST_SCN이 ETH_EmergencyAlert를 validation ingress stimulus로 emit할 수도 있습니다.
- 현재 executable baseline에서는 V2X가 emergency ingress를 validation injector path에서만 받고, ADAS도 ethObjectRiskInputMsg를 validation injector path에서만 받습니다.
- 그렇다고 TEST_SCN이 product ECU owner가 되는 것은 아닙니다.

4.5 Health와 timeout 상태는 boundary authority가 소유한다 health와 degradation state는 ad-hoc local guess가 아니라 boundary authority path에서 나와야 합니다.

대표 seam:

- Core::timeoutClear
- CoreState::warningPathStatus
- CoreState::e2eHealthState
- CoreState::selectedAlertEffectiveLevel
- CoreState::selectedAlertEffectiveType
- CoreState::selectedAlertGateReason
- CoreState::driverReleaseReason
- V2X::ingressHeartbeat
- ethFailSafeStateMsg

0.8.5 5. 사용 규칙

- seam owner를 판단할 때는 owner-route와 함께 읽습니다.
- cross-domain visibility 여부는 multibus-policy와 함께 읽습니다.
- frame/message 소유권과 source bus는 communication-matrix와 함께 읽습니다.

0.9 Multi-Bus Visibility Policy

원문:

- ../../contracts/multibus-policy.md

동기화 기준:

- 5d83ee7f
- node 이름, DBC 이름, bus 이름은 canonical technical identifier로 유지합니다.

Important

이 문서는 active SIL baseline에서 어떤 node가 추가 bus/database visibility를 가져야 하는지 정리한 핵심 계약 문서입니다. GUI 배치, source/mirror 배치, 실제 runtime 참여 여부를 같은 뜻으로 읽으면 안 됩니다.

0.9.1 1. 목적

이 문서는 active CANoe SIL configuration에서 node별 multibus visibility 필요 여부를 정의합니다.

이 문서가 답하는 질문은 아래와 같습니다.

- 어떤 node가 자기 primary domain 밖의 CAN database를 추가로 봐야 하는가
- 어떤 node가 GUI Ethernet runtime placement까지 유지해야 하는가
- 어떤 node는 source/mirror에서는 ETH_Backbone 아래에 남되, GUI Ethernet runtime은 제거해야 하는가

0.9.2 2. 핵심 원칙

1. visible surface node는 OEM-style ECU 이름을 유지합니다.
2. 추가 bus/database visibility가 필요하다고 해서 별도의 top-level gateway ECU를 새로 만드는 것은 아닙니다.
3. Primary source/mirror placement, GUI Ethernet runtime placement, extra CAN visibility는 서로 다른 판단 축입니다.
4. multibus assignment는 old backbone stub seam이 아니라 실제 foreign-domain CAN message visibility 필요성으로 결정합니다.
5. direct Ethernet RX ownership은 현재 baseline에서 CGW, ADAS, V2X, TEST_BAS로 제한합니다.
6. owner, observer, validation 예외는 contracts/layer-separation-policy.md 기준으로 해석합니다.

0.9.3 3. GUI 해석 규칙

- Ethernet runtime placement in GUI = required
  - GUI에서 ETH_Backbone을 Assigned buses에 유지합니다.
  - 대응 TCP/IP stack interface도 함께 유지합니다.
- Ethernet runtime placement in GUI = not required
  - GUI에서 ETH_Backbone을 Assigned buses에서 제거합니다.
  - IPv4만 비우고 node를 backbone에 매단 채 두는 방식으로 처리하면 안 됩니다.

0.9.4 4. Multibus가 필요한 이유

4.1 Cross-domain runtime dependency

- 어떤 node는 다른 domain database에서 생산된 message를 소비합니다.
- 어떤 node는 자기 primary domain이 아닌 foreign-domain CAN database로 publish합니다.

- 이런 node는 visible owner는 하나로 유지하되, GUI visibility는 추가로 필요합니다.

4.2 Full-system validation harness

- TEST_SCN은 cross-domain scenario를 주입하고 관찰해야 합니다.
- 따라서 Powertrain, Chassis, Body, Infotainment, ADAS domain CAN을 모두 볼 수 있어야 합니다.
- 이는 product transport 설계가 아니라 validation architecture 요구사항입니다.

0.9.5 5. Node 분류

5.1 진성 multibus anchor

Node	이유
CGW	cross-domain boundary와 fail-safe authority를 가지며 frmChassisHealthMsg, frmBodyHealthMsg,
TEST_SCN	full-system scenario orchestration을 수행하며 five-domain CAN contract를 모두 주입/관찰해야

5.2 CAN-primary이지만 cross-domain visibility가 필요한 node

Node	Primary placement	GUI Ethernet runtime	추가 CAN visibility	핵심 이유
PGS	Infotainment	not required	ADAS	frmParkUltras
AFLS	Body	not required	Chassis	frmSteeringA
DATC	Body	not required	Infotainment	frmTmuServic
ACU	Chassis	not required	Body	frmSeatBeltSt
ODS	Chassis	not required	Body	frmSeatBeltSt
ADAS	ADAS	required	Chassis, Body, Infotainment, Powertrain	cross-domain
BCM	Body	not required	Chassis, Infotainment, ADAS	frmVehicleSta
IVI	Infotainment	required	Chassis, ADAS	frmVehicleSta
SCC	ADAS	not required	Powertrain, Chassis	frmCruiseStat
HWP	ADAS	not required	Powertrain	frmCruiseStat
VCU	Chassis	required	Powertrain, Infotainment	frmIgnitionEn
MDPS	Chassis	required	none	foreign CAN
CLU	Infotainment	not required	ADAS	frmAdasDoma

5.3 ETH_Backbone-primary node

Node	Primary placement	GUI Ethernet runtime	추가 CAN visibility	핵
CGW	ETH_Backbone	required	Chassis, Body, Infotainment	he
TEST_SCN	ETH_Backbone	required	Powertrain, Chassis, Body, Infotainment, ADAS	va
V2X	ETH_Backbone	required	none	er
DCM	ETH_Backbone	not required	Infotainment	sc
ETHB	ETH_Backbone	not required	Infotainment	in
SGW	ETH_Backbone	not required	Chassis, Infotainment	fr
IBOX	ETH_Backbone	not required	Chassis, Infotainment, ADAS	ve
EDR	ETH_Backbone	not required	ADAS	ob
TEST_BAS	ETH_Backbone	required	none	sh

0.9.6 6. TEST_BAS 단일-bus 유지 규칙

TEST_BAS는 TEST_SCN과 다르게 full-system stimulus를 만들지 않습니다.

- TEST_SCN
  - domain CAN과 backbone contract를 직접 주입/관찰합니다.
- TEST_BAS
  - Test::scenarioResult, Test::base* 같은 sysvar summary seam을 집계합니다.

따라서 TEST_BAS는 backbone runtime에는 남지만, foreign CAN database visibility를 무조건 늘릴 필요는 없습니다.

0.9.7 7. 사용 규칙

- source/mirror 구조를 바꾸기 전에 이 문서로 visibility 필요성을 먼저 판단합니다.
- GUI Assigned buses 변경은 이 문서의 GUI Ethernet runtime 판정에 따라 수행합니다.
- foreign-domain CAN visibility가 필요하면 communication-matrix와 owner-route에서 실제 owner/seam을 함께 확인합니다.

0.10 Panel and SysVar Binding Contract

원문:

- ../../contracts/panel-sysvar-contract.md

동기화 기준:

- 5d83ee7f
- SysVar namespace와 binding target 이름은 canonical technical identifier로 유지합니다.

Important

이 문서는 panel input, system-variable namespace, runtime-visible output monitor 사이의 stable binding contract를 정의합니다. panel editor의 클릭 절차가 아니라, 무엇을 쓰고 읽어야 하는지에 대한 계약 문서입니다.

0.10.1 1. 목적

이 문서는 아래 항목 사이의 stable binding contract를 정의합니다.

- CANoe panel input
- system-variable namespace
- runtime-visible output monitor

핵심 목적은 두 가지입니다.

1. panel이 어떤 namespace에 써도 되는지 고정한다.
2. panel이 어떤 monitor namespace를 읽어야 하는지 고정한다.

0.10.2 2. Active binding 원칙

1. panel은 stable system-variable 이름에 바인딩해야 합니다.
2. temporary alias나 transport-specific internal에 새 widget을 직접 연결하면 안 됩니다.
3. panel source of truth는 cfg/channel_assign/**가 아니라 이 binding contract입니다.
4. 입력 widget은 approved input namespace에만 씁니다.
5. 관찰 widget은 approved monitor namespace에서만 읽습니다.

0.10.3 3. Write surface

3.1 Chassis 입력 binding

SysVar	의미	방향
Chassis::vehicleSpeed	panel input speed	panel writes
Chassis::driveState	drive selector state	panel writes
Chassis::steeringInput	steering activity flag	panel writes

3.2 Infotainment 입력 binding

SysVar	의미	방향
Infotainment::roadZone	road-zone context	panel writes
Infotainment::navDirection	navigation direction	panel writes
Infotainment::zoneDistance	zone boundary까지의 거리	panel writes
Infotainment::speedLimit	필요 시 speed-limit context	panel writes 또는 scripted input

3.3 V2X 입력 binding

SysVar	의미	방향
V2X::emergencyType	emergency vehicle type	panel reads
Test::compatEmergencyDirection	dispatch-only compat 방향 입력	panel writes
Test::compatEmergencyEta	dispatch-only compat ETA 입력	panel writes
Test::compatEmergencySourceId	dispatch-only compat source identifier 입력	tie-break 검증 시 panel writes
V2X::alertState	emergency active/clear state	panel reads
Test::compatPoliceDispatch	police dispatch compat toggle	dispatch flow 검증 시 panel w
Test::compatAmbulanceDispatch	ambulance dispatch compat toggle	dispatch flow 검증 시 panel w

3.4 Test control binding

SysVar	의미	방향
Test::testScenario	selected SIL scenario id	panel 또는 automation writes
Test::scenarioCommand	execute command	panel 또는 automation writes
Test::forceFailSafe	validation override	panel 또는 automation writes
Test::displayModeSetting	test-only HMI mode override	panel 또는 automation writes
Test::alertVolumeSetting	test-only audio override	panel 또는 automation writes
Test::compatPoliceDispatch	dispatch-only compatibility stimulus	panel 또는 automation writes
Test::compatAmbulanceDispatch	dispatch-only compatibility stimulus	panel 또는 automation writes
Test::compatEmergencyDirection	dispatch-only compatibility direction	panel 또는 automation writes
Test::compatEmergencyEta	dispatch-only compatibility ETA	panel 또는 automation writes
Test::compatEmergencySourceId	dispatch-only compatibility source id	panel 또는 automation writes

0.10.4 4. Read surface

4.1 Primary product-visible output

SysVar	의미	방향
Body::ambientMode	ambient output mode	panel reads
Body::ambientColor	ambient output color	panel reads
Body::ambientPattern	ambient output pattern	panel reads
Cluster::warningTextCode	cluster warning result	panel reads

4.2 Runtime status mirror

SysVar	의미	방향
Core::selectedAlertLevel	selected alert level	panel reads
Core::selectedAlertType	selected alert type	panel reads
Core::timeoutClear	timeout clear state	panel reads
Core::proximityRiskLevel	emergency proximity risk	panel reads
Core::decelAssistReq	deceleration assist request	panel reads
Core::failSafeMode	fail-safe mode	panel reads

4.3 Health 및 traceability mirror

SysVar	의미	방향
CoreState::warningPathStatus	warning-path health state	panel reads
CoreState::e2eHealthState	end-to-end health state	panel reads
CoreState::domainBoundaryStatus	boundary alive summary	panel reads
CoreState::lastEmergencyRxMs	emergency receive timestamp mirror	panel reads
CoreState::alertHistoryCount	alert history accumulation	panel reads

4.4 Verification 및 render mirror

SysVar	의미	방향
Test::scenarioResult	per-scenario PASS/FAIL summary	panel reads
Test::baseScenarioResult	baseline aggregate PASS/FAIL	panel reads
Diag::*	diagnostic observation seam	panel 또는 tools read
UiRender::*	presentation-only derived render state	demo rendering 사용 시 panel reads

0.10.5 5. Binding 규칙

1. 입력 widget은 input namespace에만 씁니다.
2. product output monitor는 stable output namespace를 우선 읽습니다.
3. g* alias나 temporary debug var에 새 widget을 직접 묶지 않습니다.
4. panel은 transport-specific detail보다 reviewer-readable namespace를 우선 보여줘야 합니다.
5. binding contract가 바뀌면 panel README와 0304 추적도 함께 갱신해야 합니다.

0.11 Ethernet Backbone Interface Specification 안내본

원문:

- ../../contracts/ethernet-backbone.md

상태:

- 이 문서는 canonical 원문이 이미 한국어 중심으로 작성되어 있습니다.
- 따라서 본 안내본은 별도 번역 복제본이 아니라 한글팩 구조 안에서 canonical 문서로 연결하기 위한 진입점입니다.

0.11.1 이 문서에서 확인할 것

- active Ethernet backbone contract의 단일 원본
- UDP multicast 239.0.2.1:5000 기준
- 0x510/0x511/0x512, 0xE100, 0xE200, 0xE210~0xE216, 0x1C2의 signal contract
- validation result sysvar seam
- 0302 -> 0303 -> 0304 -> 04 -> 05/06/07 업데이트 순서

0.11.2 바로 가기

- canonical 문서 열기: ../../contracts/ethernet-backbone.md
- 관련 interface 문서: ../../contracts/ethernet-interface.md
- 관련 matrix 문서: ./communication-matrix.md

S3. 검증 계약 세트

본 별첨은 시험 판정 기준, acceptance criteria, executable asset mapping, execution guide를 제시하여 논문의 검증 구조를 보강한다.

원문 기준 경로

- canoe/docs/Kor/verification/oracle.md
- canoe/docs/Kor/verification/acceptance-criteria.md
- canoe/docs/Kor/verification/test-asset-mapping.md
- canoe/docs/Kor/verification/execution-guide.md
- canoe/docs/Kor/verification/evidence-policy.md

0.12 Oracle

원문:

- ../../verification/oracle.md

동기화 기준:

- 5d83ee7f
- oracle layer 이름, SysVar 이름, contract 문서 이름은 canonical technical identifier로 유지합니다.

Important

이 문서는 CANoe SIL baseline이 올바른지 판정하는 layered oracle model을 정의합니다. PASS는 단일 signal이 아니라 contract, behavior, harness, evidence가 함께 맞아야 성립합니다.

0.12.1 목적

이 문서는 CANoe SIL baseline의 정합성을 판정할 때 사용하는 oracle model을 정의합니다.

oracle은 단일 signal check가 아닙니다. 아래 네 계층이 모두 일치해야 true PASS로 봅니다.

- contract
- behavior
- harness
- evidence

0.12.2 Oracle 모델

Oracle 계층	주 소스	PAS
Contract oracle	communication matrix, owner/route contract, multibus policy, interface contract	runt
ECU behavior oracle	ECU classification, panel/sysvar contract, diagnostic-sysvar contract	각 a
Scenario oracle	acceptance criteria	scen
Harness oracle	TEST_SCN, TEST_BAS, native Test Unit verdict	nati
Evidence oracle	evidence policy와 captured artifact	evid

0.12.3 Oracle source

- ../architecture/ecu-classification.md
- ../contracts/communication-matrix.md

- ../contracts/owner-route.md
- ../contracts/multibus-policy.md
- ../contracts/panel-sysvar-contract.md
- ../contracts/diagnostic-sysvar-contract.md
- acceptance-criteria.md
- execution-guide.md
- evidence-policy.md

0.12.4 현재 판정 규칙

검증 항목이 true PASS가 되려면 아래가 모두 참이어야 합니다.

1. runtime이 문서화된 contract path를 따를 것
2. ECU 또는 surface behavior가 문서화된 ownership boundary 안에 있을 것
3. scenario result가 acceptance criteria와 일치할 것
4. harness verdict가 그 기대 결과와 일치할 것
5. 필요한 evidence가 존재하고 review 가능할 것

즉 harness verdict만 PASS라고 해서 전체 PASS가 되는 것은 아닙니다.

예:

- verdict는 PASS인데 contract path가 문서화되지 않음
- verdict는 PASS인데 required evidence가 없음
- 출력은 한 번 맞아 보이지만 repeatability 또는 timeout rule이 실패함

0.12.5 Harness 기반 oracle 해석

현재 baseline이 쓰는 핵심 seam은 아래와 같습니다.

Harness seam	해석
Test::scenarioResult	TEST_SCN이 기록하는 scenario-level PASS/FAIL
Test::baseScenarioResult	TEST_BAS가 기록하는 aggregate baseline verdict
Test::baseFlowCoverageMask	review completeness를 위한 coverage summary
Test::baseTraceSnapshotId	evidence navigation용 trace anchor
Test::baseTestHealth	harness trustworthiness를 판단하는 health summary

0.12.6 현재 oracle 경계

active oracle은 아래 범위에 한정합니다.

- CANoe SIL
- CAN + Ethernet runtime seam
- 현재 문서화된 product surface와 validation surface

이 문서는 미래의 모든 ECU와 모든 customer test case가 완전히 커버되었다고 주장하지 않습니다.

0.12.7 개발 메모

향후 프로젝트 전체 test architecture가 customer workproduct chain 기준으로 다시 정리되더라도, 아래 구조는 유지해야 합니다.

- layered oracle structure
- product behavior와 validation behavior의 분리
- PASS는 contract, behavior, harness, evidence가 모두 동의해야 한다는 원칙

0.13 Acceptance Criteria

원문:

- ../../verification/acceptance-criteria.md

동기화 기준:

- 5d83ee7f
- scenario ID, PASS condition 식, signal 이름은 canonical technical string으로 유지합니다.

Important

이 문서는 active baseline에 대한 current CANoe SIL scenario-level PASS/FAIL criteria를 정의합니다. TEST_SCN verdict logic과 TEST_BAS aggregation flow를 읽을 때 기준이 되는 핵심 문서입니다.

0.13.1 목적

이 문서는 active baseline의 scenario-level PASS/FAIL 기준을 정의합니다.

판정 기준은 두 축에 맞춰 읽습니다.

- 현재 TEST_SCN verdict logic
- 현재 TEST_BAS baseline aggregation flow

0.13.2 Scenario verdict 표

ID	Scenario	Requirement refs	PASS condition
S01	Idle normal	Req_001, Req_002	selectedAlertLevel==0 AND wa
S02	School-zone overspeed	Req_009, Req_010, Req_037	level==3 AND type==3 AND am
S03	Highway steering inactivity	Req_011, Req_012, Req_038	level==2 AND type==4 AND am
S04	Police emergency	Req_017, Req_022, Req_035, Req_036	level==6 AND type==1 AND cod
S05	Ambulance emergency	Req_018, Req_022, Req_035, Req_036	level==7 AND type==2 AND cod
S06	Timeout clear	Req_024	timeoutClear==1 AND level==0
S07	Guided section left	Req_013, Req_014, Req_039	level==2 AND type==5 AND am
S08	Guided section right	Req_013, Req_014, Req_039	level==2 AND type==6 AND am
S09	ETA priority	Req_030	V2X::eta==10 AND V2X::source
S10	Source ID tie-break	Req_031	V2X::sourceId==5 AND V2X::eta
S11	Transition moderation	Req_034	level==6 AND type==1 AND sna
S12	Duplicate warning suppression	Req_006, Req_026	level==3 AND type==3 AND dup
S13	Section-transition stability	Req_015	level==3 AND type==3 AND sna
S14	Domain boundary integrity	Req_110, Req_111, Req_112	domainBoundaryStatus==1 AND

0.13.3 Verdict 흐름

```
Test::testScenario = N
 -> applyScenarioPreset(N)
 -> scheduleScenarioEvaluation(N)
 -> evaluateScenarioResult()
 -> Test::scenarioResult = PASS / FAIL
 -> TEST_BAS aggregates Test::scenarioResult
 -> Test::baseScenarioResult and Test::baseTestHealth are updated
```

즉 scenario-level verdict와 baseline aggregate verdict를 구분해서 읽어야 합니다.

0.13.4 현재 oracle signal

Signal 또는 변수	역할
Test::scenarioResult	현재 scenario 단위 PASS/FAIL
Test::baseScenarioId	baseline aggregation target
Test::baseScenarioResult	aggregate PASS/FAIL 결과
Test::baseFlowCoverageMask	aggregate coverage summary
Test::baseTraceSnapshotId	aggregate trace anchor
Test::baseTestHealth	aggregate harness health
Core::selectedAlertLevel	최종 선택 경고 레벨
Core::selectedAlertType	최종 선택 경고 타입
Cluster::warningTextCode	reviewer-visible warning text 결과
Body::ambientColor	ambient output color
Body::ambientPattern	ambient output pattern
CoreState::duplicatePopupGuard	duplicate-warning suppression state
CoreState::arbitrationSnapshotId	arbitration transition counter
CoreState::domainBoundaryStatus	boundary health state
CoreState::routingPolicy	routing policy state
V2X::eta	active emergency ETA
V2X::sourceId	active emergency source identifier

0.13.5 Output-style reference

Alert type	의미	Ambient color	Ambient pattern
emergency police	police emergency warning	6	6
emergency ambulance	ambulance emergency warning	7	7
school-zone	school-zone warning	3	5
highway steering	highway steering warning	4	4
guide left	directional guidance left	5	2
guide right	directional guidance right	5	3
generic normal	nominal state	1	1
none	no warning	0	0

0.13.6 개발 메모

이 문서는 현재 executable baseline 기준입니다. 향후 01, 05, 06, 07을 기준으로 full architecture가 다시 정리되면 이 문서도 scenario-level acceptance baseline으로 함께 갱신해야 합니다.

0.14 Test Asset Mapping

원문:

- ../../verification/test-asset-mapping.md

동기화 기준:

- 5d83ee7f
- native asset 이름, oracle seam 이름, evidence artifact 이름은 canonical technical string으로 유지합니다.

Important

이 문서는 CANoe SIL test construction의 현재 개발 기준선입니다. native CANoe test asset, oracle seam, evidence collection이 최종 고정되기 전까지 일부 내용은 변경될 수 있습니다.

0.14.1 1. 목적

이 문서는 아래 reviewer-facing test ID와:

- driving-alert-workproducts/05_Unit_Test.md
- driving-alert-workproducts/06_Integration_Test.md
- driving-alert-workproducts/07_System_Test.md

다음 실행 자산 사이의 공식 매핑을 정의합니다.

- native CANoe test asset 후보
- primary oracle source
- primary evidence source
- 현재 diagnostic 필요 여부

즉, 공식 05/06/07 표와 executable CANoe SIL test surface를 연결하는 implementation-side verification bridge입니다.

0.14.2 2. 매핑 규칙

1. 05는 제품 기준선의 controller, input, output verification으로 매핑합니다.
2. 06은 runtime boundary를 넘는 integrated function verification으로 매핑합니다.
3. 07은 end-user 및 whole-vehicle scenario verification으로 매핑합니다.
4. Diagnostic Needed?는 Yes와 No만 사용합니다.
5. Yes는 명시적인 diagnostic 또는 security state visibility가 없으면 verdict 근거가 약해지는 row에만 사용합니다.
6. native asset 후보, oracle type, evidence type이 완전히 동일한 simulator preset row만 반복 row로 묶을 수 있습니다.

추가 해석 원칙:

- asset ID, scenario ID, oracle seam, evidence artifact 이름은 canonical technical string으로 유지합니다.
- 대형 표 안에서는 reviewer가 바로 판단해야 하는 의미를 우선 설명하고, raw implementation detail은 줄입니다.
- 표 셀 안에 영어 technical phrase가 남아 있더라도 그것은 식별자 보존을 위한 것이며, 해석 기준은 한국어 section과 note를 우선 봅니다.

0.14.3 3. Unit Test 매핑

3.1 공통 oracle 표현 읽기 대형 표 안에서 반복되는 oracle 표현은 아래처럼 해석합니다.

표기	한국어 해석
selected warning state	최종 선택 경고 상태
zone context state	구역 판단 상태
clear and restore state	해제 및 복원 상태
final selected warning state	최종 선택 경고 결과
fail-safe state	fail-safe 진입 또는 유지 상태
minimum-channel state	최소 채널 유지 상태
display-policy reflection state	표시 정책 반영 상태
audio focus, ducking, and volume policy state	오디오 focus, ducking, volume 정책 상태
output-channel availability and fallback state	출력 채널 가용성과 fallback 상태
duplicate-popup suppression state	중복 팝업 억제 상태
channel-restore consistency state	채널 복원 일관성 상태
end-to-end scenario verdict	종단 간 시나리오 판정
end-to-end fail-safe scenario verdict	종단 간 fail-safe 시나리오 판정

3.2 공통 evidence 표현 읽기 evidence column은 artifact 이름을 유지하되, 조합 의미는 아래처럼 읽습니다.

표기	한국어 해석
native report + trace + sysvar snapshot	native report, trace, sysvar snapshot을 함께 보는 기본 조
native report + sysvar snapshot + panel capture	runtime report와 sysvar, panel capture를 함께 보는 조합
panel capture + screenshot + native report	panel 기준의 visible output evidence 조합
cluster capture + screenshot + native report	cluster/HUD 기준 visual evidence 조합
write window + trace + sysvar snapshot	diagnostic 또는 fail-safe 확인용 심화 evidence 조합
Ethernet trace + write window + verification_log.csv	외부 TX/주기 확인용 transport evidence 조합
panel capture + trace + verification_log.csv	system-level 동작과 trace chronology를 함께 보는 조합

ID	후보 native asset	주요 oracle
UT_001	TC_CANOE_UT_CORE_001_CGW_CHS_GW	vehicle-state 전달 seam
UT_002	TC_CANOE_UT_CORE_002_CGW_INFOTAINMENT_GW	navigation 맥락 전달 seam
UT_003	TC_CANOE_UT_CORE_003_CGW_BOUNDARY_STATUS	선택 경고 상태
UT_004	TC_CANOE_UT_CORE_004_V2X_EVENT_MAINTAIN	emergency 상태 및 timeout clear
UT_005	TC_CANOE_UT_CORE_005_ADAS_DECEL_ASSIST	arbitration 결과 및 decel-assist 상태
UT_006	TC_CANOE_UT_EXT_006_OBJECT_RISK	object-risk 상태 및 downgrade 상태
UT_007	TC_CANOE_UT_EXT_007_CLU_CONTEXT_ADJUST	렌더링된 경고 맥락
UT_008	TC_CANOE_UT_EXT_008_DOMAIN_BOUNDARY_FAILSAFE	fail-safe 진입 상태 및 boundary-health
UT_009	TC_CANOE_UT_CORE_009_NAV_CTX_MGR	구역 맥락 상태
UT_010	TC_CANOE_UT_CORE_010_EMS_ALERT_TXRX	emergency tx/rx 상태 및 timeout 상태
UT_011	TC_CANOE_UT_CORE_011_ADAS_WARNING_SELECTION	최종 선택 경고 상태
UT_012	TC_CANOE_UT_CORE_012_BODY_GW_ROUTE	ambient route 상태
UT_013	TC_CANOE_UT_CORE_013_IVI_GW_ROUTE	cluster route 상태
UT_014	TC_CANOE_UT_CORE_014_BCM_AMBIENT_POLICY	ambient 색상 및 패턴 상태
UT_015	TC_CANOE_UT_CORE_015_IVI_TEXT_MAPPING	경고 text 및 direction 상태

ID	후보 native asset	주요 oracle
UT_016	TC_CANOE_UT_EXT_016_CHS_BRAKE_EXT	brake 관련 맥락 상태
UT_017	TC_CANOE_UT_EXT_017_CHS_DYNAMICS_EXT	chassis 맥락 상태
UT_018	TC_CANOE_UT_EXT_018_BODY_ENTRY_EXIT	door/tailgate 맥락 상태
UT_019	TC_CANOE_UT_EXT_019_BODY_OCCUPANT_PROTECTION	occupant-protection 맥락 상태
UT_020	TC_CANOE_UT_EXT_020_BODY_COMFORT	comfort 맥락 상태
UT_021	TC_CANOE_UT_EXT_021_IVI_DISPLAY_SERVICE	표시/서비스 맥락 상태
UT_022	TC_CANOE_UT_EXT_022_IVI_SERVICE_ACCESS	service-access 맥락 상태
UT_023	TC_CANOE_UT_EXT_023_ADAS_DRIVE_ASSIST	drive-assist 맥락 상태
UT_024	TC_CANOE_UT_EXT_024_ADAS_PARKING_PERCEPTION	parking/perception 맥락 상태
UT_025	TC_CANOE_UT_EXT_025_WARNING_DELIVERY_BOUNDARY	delivery-health 및 fail-safe 상태
UT_026	TC_CANOE_UT_EXT_026_DOMAIN_ROUTER_PROPULSION	propulsion 맥락 상태
UT_027	TC_CANOE_UT_EXT_027_DOMAIN_ROUTER_POWER_CHARGE	power/charge 맥락 상태
UT_028	TC_CANOE_UT_INP_028_VEHICLE_STEERING	drive-state / speed / steering 정규화
UT_029	TC_CANOE_UT_INP_029_NAV_CONTEXT	nav 맥락의 alert-level trigger 및 안정
UT_030	TC_CANOE_UT_INP_030_EMERGENCY_INPUT	emergency type / ETA 관측 패턴
UT_031	TC_CANOE_UT_INP_031_EPB_INPUT	park brake decel 입력 정규화
UT_032	TC_CANOE_UT_INP_032_EHB_INPUT	hydraulic brake decel 입력 정규화
UT_033	TC_CANOE_UT_INP_033_VSM_INPUT	VSM 안정 상태 및 spurious alert 없음
UT_034	TC_CANOE_UT_INP_034_ECS_INPUT	air suspension ride-height mode
UT_035	TC_CANOE_UT_INP_035_CDC_INPUT	damper mode and valve current
UT_036	TC_CANOE_UT_INP_036_DOOR_FL_INPUT	front-left door auto unlock / approach s
UT_037	TC_CANOE_UT_INP_037_DOOR_FR_INPUT	front-right door auto unlock / approach
UT_038	TC_CANOE_UT_INP_038_DOOR_RL_INPUT	rear-left door auto unlock / approach s
UT_039	TC_CANOE_UT_INP_039_DOOR_RR_INPUT	rear-right door auto unlock / approach
UT_040	TC_CANOE_UT_INP_040_TGM_INPUT	tailgate assist mode and actuator comm
UT_041	TC_CANOE_UT_INP_041_ACU_INPUT	airbag deployment armed + pretension
UT_042	TC_CANOE_UT_INP_042_ODS_INPUT	occupant detection / weight class / chil
UT_043	TC_CANOE_UT_INP_043_AFLS_INPUT	adaptive front-light mode + headlamp a
UT_044	TC_CANOE_UT_INP_044_AHLS_INPUT	auto high-beam assist + active flag
UT_045	TC_CANOE_UT_INP_045_DATC_INPUT	cabin temp / blower / AC / vent mode
UT_046	TC_CANOE_UT_INP_046_SEAT_DRV_INPUT	driver seat position + heat level
UT_047	TC_CANOE_UT_INP_047_SEAT_PASS_INPUT	passenger seat position + heat level
UT_048	TC_CANOE_UT_INP_048_SRF_INPUT	sunroof position state
UT_049	TC_CANOE_UT_INP_049_HUD_INPUT	HUD mode + warning code
UT_050	TC_CANOE_UT_INP_050_AMP_INPUT	audio mute / ducking / volume level
UT_051	TC_CANOE_UT_INP_051_TMU_INPUT	telematics link state / service mode / re
UT_052	TC_CANOE_UT_INP_052_SCC_INPUT	SCC proximity decel — decel-assist obs
UT_053	TC_CANOE_UT_INP_053_PGS_INPUT	parking guidance 활성 / 기동 준비
UT_054	TC_CANOE_UT_INP_054_PUS_INPUT	parking ultrasonic sensor proximity clas
UT_055	TC_CANOE_UT_INP_055_AVM_INPUT	surround-view mode and active flag
UT_056	TC_CANOE_UT_INP_056_FCAM_INPUT	forward camera health and lane preview
UT_057	TC_CANOE_UT_INP_057_FRADAR_INPUT	forward radar object range and risk clas
UT_058	TC_CANOE_UT_INP_058_SRR_FL_INPUT	left-front SRR — intersection conflict fl

ID	후보 native asset	주요 oracle
UT_059	TC_CANOE_UT_INP_059_SRR_FR_INPUT	right-front SRR — merge cut-in flag
UT_060	TC_CANOE_UT_INP_060_SRR_RL_INPUT	rear-left SRR blind-spot state
UT_061	TC_CANOE_UT_INP_061_SRR_RR_INPUT	rear-right SRR blind-spot state
UT_062	TC_CANOE_UT_INP_062_IBOX_INPUT	digital key / vehicle service 상태
UT_063	TC_CANOE_UT_EXT_063_SGW_SECURITY_STATE	security-state injection
UT_064	TC_CANOE_UT_EXT_064_DCM_DIAGNOSTIC_STATE	diagnostic-state injection
UT_065	TC_CANOE_UT_INP_065_ETHB_INPUT	backbone failure 및 fail-safe 관측
UT_066	TC_CANOE_UT_INP_066_OBC_INPUT	OBC charge power and AC plug state
UT_067	TC_CANOE_UT_INP_067_DCDC_INPUT	DCDC LV output voltage and current
UT_068	TC_CANOE_UT_INP_068_MCU_INPUT	motor torque command and speed rpm
UT_069	TC_CANOE_UT_INP_069_INVERTER_INPUT	inverter state and DC link voltage
UT_070	TC_CANOE_UT_OUT_070_BCM_AMBIENT	ambient output 렌더링 상태
UT_071	TC_CANOE_UT_OUT_071_IVI_HMI	HMI output 상태
UT_072	TC_CANOE_UT_OUT_072_CLU_DISPLAY	cluster display 렌더링 상태
UT_073	TC_CANOE_UT_OUT_073_HUD_DISPLAY	HUD 렌더링 상태
UT_074	TC_CANOE_UT_OUT_074_AMP_AUDIO	audio-guide 상태
UT_075	TC_CANOE_UT_OUT_075_DECEL_ASSIST_REQ	decel-assist request 상태
UT_076	TC_CANOE_UT_OUT_076_POLICE_TX	external tx frame 관찰
UT_077	TC_CANOE_UT_OUT_077_AMBULANCE_TX	external tx frame 관찰

0.14.4 4. Integration Test 매핑

ID	후보 native asset	주요 oracle
IT_001	TC_CANOE_IT_CORE_001_BASE_ACTIVATION	경고 활성/비활성 상태
IT_002	TC_CANOE_IT_CORE_002_SCHOOLZONE_PATH	구역 경고 결과 상태
IT_003	TC_CANOE_IT_CORE_003_HIGHWAY_NOSTEER_PATH	고속도로 경고 발생 및 해제 상태
IT_004	TC_CANOE_IT_V2_004_POLICE_RX	경찰차 경고 결과 상태
IT_005	TC_CANOE_IT_V2_005_AMBULANCE_RX	구급차 경고 결과 상태
IT_006	TC_CANOE_IT_V2_006_ARBITRATION	최종 선택 경고 상태
IT_007	TC_CANOE_IT_CORE_007_AMBIENT_OUTPUT	ambient 색상/패턴 결과
IT_008	TC_CANOE_IT_CORE_008_CLUSTER_DIRECTION_OUTPUT	cluster 텍스트/방향 결과
IT_009	TC_CANOE_IT_V2_009_TIMEOUT_CLEAR	해제 및 복원 상태
IT_010	TC_CANOE_IT_V2_010_DECEL_ASSIST	decel-assist request 및 동기 상태
IT_011	TC_CANOE_IT_V2_011_FAILSAFE_MIN_WARNING	fail-safe 및 최소 채널 상태
IT_012	TC_CANOE_IT_EXT_012_OBJECT_RISK_EVENTLOG	object-risk 및 event-log 상태
IT_013	TC_CANOE_IT_013_SEATBELT_CONTEXT	안전벨트 강조 상태
IT_014	TC_CANOE_IT_014_DISPLAY_POLICY	표시 정책 반영 상태
IT_015	TC_CANOE_IT_015_TURN_LAMP_CONTEXT	turn-lamp 맥락에 따른 경고 유형 조정
IT_016	TC_CANOE_IT_016_DRIVE_MODE_SENSITIVITY	주행 모드 민감도 상태
IT_017	TC_CANOE_IT_017_AUDIO_VOLUME_POLICY	오디오 focus, ducking, volume 정책 상태
IT_018	TC_CANOE_IT_EXT_018_EMERGENCY_PLUS_TTC	경고/decel 결합 결과 상태
IT_019	TC_CANOE_IT_019_POWERTRAIN_PARKED_BASELINE	주차 기준선 상태
IT_020	TC_CANOE_IT_020_POWERTRAIN_DRIVE_BASELINE	주행 기준선 상태

ID	후보 native asset	주요 oracle
IT_021	TC_CANOE_IT_021_CHASSIS_STEERING_BASELINE	조향 입력 기준선 상태
IT_022	TC_CANOE_IT_022_CHASSIS_BRAKE_BASELINE	제동 입력 기준선 상태
IT_023	TC_CANOE_IT_023_CHASSIS_ACCEL_BASELINE	가속 기반 drive-mode 상태
IT_024	TC_CANOE_IT_BASE_024_BODY_STATE	hazard 반영 기준선 상태
IT_025	TC_CANOE_IT_EXT_025_WINDOW_STATE	window 상태 반영
IT_026	TC_CANOE_IT_BASE_026_BASIC_DISPLAY_UI	기본 표시 통합 상태
IT_027	TC_CANOE_IT_BASE_027_COMFORT_CONTEXT	comfort 맥락의 정책 상태
IT_028	TC_CANOE_IT_EXT_028_BODY_CONTROL_LOCK	door lock/open 반영 상태
IT_029	TC_CANOE_IT_EXT_029_WIPER_RAIN_BASELINE	wiper/rain 기준선 반영 상태
IT_030	TC_CANOE_IT_BASE_030_BODY_SECURITY_CONTEXT	security-state service-boundary 상태
IT_031	TC_CANOE_IT_031_AUDIO_GUIDE_RUNTIME	오디오 및 voice-guide 통합 상태
IT_032	TC_CANOE_IT_032_OUTPUT_FALLBACK	출력 채널 가용성 및 fallback 상태
IT_033	TC_CANOE_IT_033_DUPLICATE_POPUP_SUPPRESSION	중복 팝업 억제 상태
IT_034	TC_CANOE_IT_034_CHANNEL_RESTORE	채널 복원 일관성 상태
IT_035	TC_CANOE_IT_EXT_035_DISTANCE_HISTORY	거리 및 이력 조회 결과
IT_036	TC_CANOE_IT_EXT_036_CHASSIS_EXT_CONTEXT	EPB/EHB/VSM/ECS/CDC 통합 상태
IT_037	TC_CANOE_IT_EXT_037_OCCUPANT_COMFORT_CONTEXT	occupant/comfort 통합 상태
IT_038	TC_CANOE_IT_EXT_038_DISPLAY_SERVICE_CONTEXT	표시/서비스 통합 상태
IT_039	TC_CANOE_IT_EXT_039_ADAS_PERCEPTION_CONTEXT	ADAS/perception 통합 상태
IT_040	TC_CANOE_IT_EXT_040_SERVICE_SECURITY_DIAG	service/security/diagnostic 통합 상태
IT_041	TC_CANOE_IT_EXT_041_CHARGE_POWER_CONTEXT	power/charge 통합 상태
IT_042	TC_CANOE_IT_EXT_042_DISPLAY_CHANNELS	채널 간 visual 일관성
IT_043	TC_CANOE_IT_EXT_043_AUDIO_GUIDE_CHANNEL	audio-guide 일관성
IT_044	TC_CANOE_IT_ETH_044_POLICE_TX	경찰 외부 TX 관찰
IT_045	TC_CANOE_IT_ETH_045_AMBULANCE_TX	구급차 외부 TX 관찰

0.14.5 5. System Test 매핑

ID	후보 native asset	주요 oracle
ST_001	TC_CANOE_ST_CORE_001_POWER_ON_BASELINE	무경고 기준선 상태
ST_002	TC_CANOE_ST_CORE_002_NORMAL_DRIVE	정상 주행 상태
ST_003	TC_CANOE_ST_CORE_003_BASIC_WARNING_ACTIVATION	기본 경고 활성 결과
ST_004	TC_CANOE_ST_CORE_004_BASIC_WARNING_CLEAR	정상 복귀 상태
ST_005	TC_CANOE_ST_CORE_005_ENTER_SCHOOL_ZONE	school-zone 경고 전이
ST_006	TC_CANOE_ST_CORE_006_EXIT_SCHOOL_ZONE	school-zone 복구 상태
ST_007	TC_CANOE_ST_CORE_007_HIGHWAY_POLICY_TRANSITION	고속도로 정책 전이
ST_008	TC_CANOE_ST_CORE_008_STEERING_INACTIVITY	경고 발생 및 해제 결과
ST_009	TC_CANOE_ST_CORE_009_GUIDE_LEFT	좌측 안내 표시 결과
ST_010	TC_CANOE_ST_CORE_010_GUIDE_RIGHT_CLEAR	우측 안내 표시 결과
ST_011	TC_CANOE_ST_V2_011_POLICE_OVERRIDE	경찰 우선 결과
ST_012	TC_CANOE_ST_V2_012_AMBULANCE_OVERRIDE	구급차 우선 결과
ST_013	TC_CANOE_ST_CORE_013_POLICE_DIRECTION_RIGHT	경찰 방향 표시
ST_014	TC_CANOE_ST_CORE_014_AMBULANCE_DIRECTION_LEFT	구급차 방향 표시

ID	후보 native asset	주요 oracle
ST_015	TC_CANOE_ST_V2_015_AMBULANCE_PRIORITY	최종 선택 경고 상태
ST_016	TC_CANOE_ST_V2_016_POLICE_TIEBREAK	ETA 및 SourceID 중재 결과
ST_017	TC_CANOE_ST_V2_017_AMBULANCE_TIEBREAK	ETA 및 SourceID 중재 결과
ST_018	TC_CANOE_ST_018_POLICE_TX_PERIOD	경찰 TX 주기 관찰
ST_019	TC_CANOE_ST_019_AMBULANCE_TX_PERIOD	구급차 TX 주기 관찰
ST_020	TC_CANOE_ST_V2_020_TIMEOUT_CLEAR	timeout 해제 및 복귀 상태
ST_021	TC_CANOE_ST_CORE_021_EMERGENCY_CLEAR_RESTORE	이전 경고 복원 상태
ST_022	TC_CANOE_ST_EXT_022_INTERSECTION_DECEL	경고/decel 결합 결과
ST_023	TC_CANOE_ST_EXT_023_MERGE_DECEL	경고/decel 결합 결과
ST_024	TC_CANOE_ST_EXT_024_DRIVER_INTERVENTION_CLEAR	운전자 개입 해제 결과
ST_025	TC_CANOE_ST_EXT_025_FAILSAFE_ENTRY	fail-safe 진입 및 최소 채널 상태
ST_026	TC_CANOE_ST_EXT_026_FAILSAFE_RECOVERY	fail-safe 복구 상태
ST_027	TC_CANOE_ST_EXT_027_FRONTAL_OBJECT_RISK	object-warning 표시 결과
ST_028	TC_CANOE_ST_EXT_028_LATERAL_OBJECT_RISK	object-warning 표시 결과
ST_029	TC_CANOE_ST_EXT_029_CUTIN_OBJECT_RISK	object-warning 표시 결과
ST_030	TC_CANOE_ST_030_SEATBELT_CONTEXT_ADJUST	안전벨트/운전자 맥락 조정 경고
ST_031	TC_CANOE_ST_031_DISTANCE_DISPLAY_CONSISTENCY	긴급 거리 표시 일관성
ST_032	TC_CANOE_ST_EXT_032_USER_SETTING_CHANGE	출력 정책 갱신 상태
ST_033	TC_CANOE_ST_EXT_033_HISTORY_QUERY	이력 조회 결과
ST_034	TC_CANOE_ST_034_DUPLICATE_POPUP_GUARD	중복 팝업 guard 안정성 결과
ST_035	TC_CANOE_ST_035_TIMEOUT_CLEAR_RESTORE	timeout-clear 복원 안정성 결과
ST_036	TC_CANOE_ST_036_FAILSAFE_RECOVERY_STABILITY	fail-safe 복구 안정성 결과
ST_037	TC_CANOE_ST_037_AUDIO_CHANNEL_STABILITY	오디오 채널 안정성 결과
ST_038	TC_CANOE_ST_038_VISUAL_CHANNEL_STABILITY	시각 채널 안정성 결과
ST_039	TC_CANOE_ST_EXT_039_CHASSIS_CONTEXT	제동/안정성 맥락 결과
ST_040	TC_CANOE_ST_EXT_040_OCCUPANT_COMFORT_CONTEXT	body/occupant/comfort 맥락 결과
ST_041	TC_CANOE_ST_EXT_041_DISPLAY_SERVICE_CONTEXT	표시/서비스 맥락 결과
ST_042	TC_CANOE_ST_EXT_042_ADAS_PERCEPTION_CONTEXT	ADAS/perception 맥락 결과
ST_043	TC_CANOE_ST_EXT_043_SERVICE_SECURITY_DIAG_CONTEXT	service/security/diagnostic 맥락 결과
ST_044	TC_CANOE_ST_EXT_044_CHARGE_POWER_CONTEXT	power/charge 맥락 결과
ST_045	TC_CANOE_ST_EXT_045_TRIP_SEQUENCE	종단 간 시나리오 판정
ST_046	TC_CANOE_ST_EXT_046_FAILSAFE_RECOVERY	종단 간 fail-safe 시나리오 판정

0.14.6 6. 현재 구현 우선순위

native asset 구축 우선순위는 다음과 같습니다.

1. UT_003, UT_009, UT_011, UT_014, UT_015
2. IT_001 to IT_008
3. ST_001 to ST_021
4. diagnostic-linked 항목:
   - UT_063
   - UT_064
   - IT_040
   - ST_043

0.14.7 7. 다른 verification 문서와의 관계

이 문서는 다음 문서와 함께 읽습니다.

- oracle.md
- execution-guide.md
- acceptance-criteria.md
- evidence-policy.md
- diagnostic-coverage.md

0.14.8 최신 diagnostic 실행 기준선 (2026-03-15)

공식 범위	Native Asset	TEST_SCN Scenario	주요 produce
UT_063	TC_CANOE_UT_EXT_063_SGW_SECURITY_STATE	203	SGW.can -> D
UT_064	TC_CANOE_UT_EXT_064_DCM_DIAGNOSTIC_STATE	204	DCM.can -> D
IT_040	TC_CANOE_IT_EXT_040_SERVICE_SECURITY_DIAG	205	SGW + DCM 통
ST_043	TC_CANOE_ST_EXT_043_SERVICE_SECURITY_DIAG_CONTEXT	202	SGW + DCM 통

0.14.9 Wave 2 direct-ownership UT 기준선

공식 범위	Native Asset	예약 TEST_SCN Scenario	현재 상태
UT_003	TC_CANOE_UT_CORE_003_CGW_BOUNDARY_STATUS	206	실행 가능한 s
UT_011	TC_CANOE_UT_CORE_011_ADAS_WARNING_SELECTION	207	실행 가능한 s
UT_014	TC_CANOE_UT_CORE_014_BCM_AMBIENT_POLICY	208	실행 가능한 s
UT_015	TC_CANOE_UT_CORE_015_IVI_TEXT_MAPPING	209	실행 가능한 s
UT_076	TC_CANOE_UT_OUT_076_POLICE_TX	4	external-TX u
UT_077	TC_CANOE_UT_OUT_077_AMBULANCE_TX	5	external-TX u

0.14.10 Wave 2 oracle 기준선

- UT_003 / 206: 정상 CGW 경계 준비 상태, 정상 routing policy, fail-safe 해제를 확인합니다.
- UT_011 / 207: 의도한 ADAS warning-selection 맥락, 정상 경고 경로, fail-safe 해제를 확인합니다.
- UT_014 / 208: 정상 body gateway health 아래의 BCM ambient 정책을 확인합니다.
- UT_015 / 209: fail-safe 우선 덮어쓰기 없이 의도한 IVI 문구/출력 매핑을 확인합니다.
- UT_076 / 4: TX 기준 oracle과 trace-gated frame 증적을 기준으로 police emergency 전송 활성화를 확인합니다.
- UT_077 / 5: TX 기준 oracle과 trace-gated frame 증적을 기준으로 ambulance emergency 전송 활성화를 확인합니다.

0.14.11 Wave 2 현재 진행 업데이트

- UT_003 / 206: 실행 가능한 scenario contract를 추가했습니다. 현재 oracle은 domainBoundaryStatus=1, routingPolicy=1, selectedAlertLevel=0, failSafeMode=0를 확인합니다.
- UT_011 / 207: 실행 가능한 scenario contract를 추가했습니다. 현재 oracle은 selectedAlertLevel=3, selectedAlertType=3, warningPathStatus=0, failSafeMode=0를 확인합니다.

- UT_014 / 208: 실행 가능한 scenario contract를 추가했습니다. 현재 oracle은 selectedAlertLevel=3, selectedAlertType=3, ambientColor=3, ambientPattern=5, failSafeMode=0를 기준으로 school-zone ambient 정책을 확인합니다.
- UT_015 / 209: 실행 가능한 scenario contract를 추가했습니다. 현재 oracle은 selectedAlertLevel=6, selectedAlertType=1, warningTextCode=101, failSafeMode=0를 기준으로 police-emergency text 매핑을 확인합니다.

0.14.12 Wave 2 진행 업데이트 (208/209)

- UT_014 / 208: 실행 가능한 scenario contract를 고정했고 현재 ambient-policy 기준선에 정렬했습니다.
- UT_015 / 209: 실행 가능한 scenario contract를 고정했고 현재 IVI text-mapping 기준선에 정렬했습니다.
- dedicated external-TX unit row는 UT_076 / 4, UT_077 / 5로 별도 추적합니다.
- UT_006: 실행 가능한 object-risk unit contract를 추가했습니다. 현재 oracle은 scenario 20, 21, 22, 24를 통해 정면/교차로/merge risk와 confidence-degrade 동작을 확인합니다.
- UT_007: 실행 가능한 CLU context-adjust unit contract를 추가했습니다. 현재 oracle은 scenario 214, 215, 222를 통해 seat-belt 강조, display-policy 반영, 거리 맥락 표시를 확인합니다.
- UT_008: 실행 가능한 boundary/fail-safe unit contract를 추가했습니다. 현재 oracle은 scenario 18, 203, 204를 통해 fail-safe 유지와 SGW/DCM service seam 해석을 확인합니다.
- UT_010: 실행 가능한 emergency lifecycle unit contract를 추가했습니다. 현재 oracle은 scenario 4, 5에서 police/ambulance 활성화를 확인하고, scenario 6에서 timeout-clear를 확인합니다.
- UT_070: 실행 가능한 BCM output unit contract를 추가했습니다. 현재 oracle은 scenario 208, 4를 통해 school-zone 및 emergency ambient 출력을 확인합니다.
- UT_071: 실행 가능한 IVI HMI output unit contract를 추가했습니다. 현재 oracle은 scenario 215, 220, 222를 통해 문구/방향, 팝업/테마, 거리 맥락을 확인합니다.
- UT_072: 실행 가능한 CLU display unit contract를 추가했습니다. 현재 oracle은 scenario 30, 33, 220을 통해 police/ambulance 방향 표시와 팝업/테마 동기화를 확인합니다.
- UT_073: 실행 가능한 HUD/front-display unit contract를 추가했습니다. 현재 oracle은 scenario 30, 220을 통해 전면 표시 공용 방향 정보와 팝업/테마 seam을 확인합니다.
- UT_074: 실행 가능한 AMP audio unit contract를 추가했습니다. 현재 oracle은 scenario 215를 통해 audio focus, ducking, volume policy를 확인합니다.
- UT_075: 실행 가능한 decel-assist output unit contract를 추가했습니다. 현재 oracle은 scenario 16, 18을 통해 활성 request와 fail-safe 차단을 확인합니다.

0.14.13 External TX unit 완료 업데이트

- UT_076 / 4: 실행 가능한 scenario contract를 추가했습니다. 현재 oracle은 V2X::alertState=1, V2X::emergencyType=1, V2X::eta=12, V2X::sourceId=11, selectedAlertLevel=6, warningTextCode=101을 기준으로 police emergency 전송 활성화를 확인합니다.
- UT_077 / 5: 실행 가능한 scenario contract를 추가했습니다. 현재 oracle은 V2X::alertState=1, V2X::emergencyType=2, V2X::eta=8, V2X::sourceId=22, selectedAlertLevel=7, warningTextCode=202를 기준으로 ambulance emergency 전송 활성화를 확인합니다.
- 이전에 UT_076/077을 차지하던 retired 맥락/표시 초안은 retire/ 아래로 이동했습니다.

0.14.14 Wave 3 system-test 기준선

공식 범위	Native Asset	TEST_SCN Scenario	현재 계약
ST_001	TC_CANOE_ST_CORE_001_POWER_ON_BASELINE	1	전원 인가 초기
ST_002	TC_CANOE_ST_CORE_002_NORMAL_DRIVE	14	정상 주행 기준
ST_003	TC_CANOE_ST_CORE_003_BASIC_WARNING_ACTIVATION	1 -> 26	general-road sin
ST_004	TC_CANOE_ST_CORE_004_BASIC_WARNING_CLEAR	26 -> 1	조건 제거 후 ba
ST_005	TC_CANOE_ST_CORE_005_ENTER_SCHOOL_ZONE	1 -> 2	정상 주행에서 s
ST_006	TC_CANOE_ST_CORE_006_EXIT_SCHOOL_ZONE	2 -> 1	정상 주행 기준
ST_007	TC_CANOE_ST_CORE_007_HIGHWAY_POLICY_TRANSITION	14 -> 244	정상 주행에서
ST_008	TC_CANOE_ST_CORE_008_STEERING_INACTIVITY	244 -> 3 -> 244	지속적인 steerin
ST_009	TC_CANOE_ST_CORE_009_GUIDE_LEFT	7	좌측 안내 warn
ST_010	TC_CANOE_ST_CORE_010_GUIDE_RIGHT_CLEAR	8 -> 1	guide-right warn
ST_011	TC_CANOE_ST_V2_011_POLICE_OVERRIDE	11	police emergenc
ST_012	TC_CANOE_ST_V2_012_AMBULANCE_OVERRIDE	223	ambulance emer
ST_013	TC_CANOE_ST_CORE_013_POLICE_DIRECTION_RIGHT	30	warningTextCod
ST_014	TC_CANOE_ST_CORE_014_AMBULANCE_DIRECTION_LEFT	33	warningTextCod
ST_015	TC_CANOE_ST_V2_015_AMBULANCE_PRIORITY	212	police와 ambula
ST_016	TC_CANOE_ST_V2_016_POLICE_TIEBREAK	10	동순위 police w
ST_017	TC_CANOE_ST_V2_017_AMBULANCE_TIEBREAK	224	동순위 ambular
ST_018	TC_CANOE_ST_018_POLICE_TX_PERIOD	4	police external t
ST_019	TC_CANOE_ST_019_AMBULANCE_TX_PERIOD	5	ambulance exte
ST_020	TC_CANOE_ST_V2_020_TIMEOUT_CLEAR	35	timeout-clear가
ST_021	TC_CANOE_ST_CORE_021_EMERGENCY_CLEAR_RESTORE	35	emergency clear
ST_027	TC_CANOE_ST_EXT_027_FRONTAL_OBJECT_RISK	20	forward TTC co
ST_028	TC_CANOE_ST_EXT_028_LATERAL_OBJECT_RISK	21	intersection con
ST_029	TC_CANOE_ST_EXT_029_CUTIN_OBJECT_RISK	22	merge conflict 조
ST_030	TC_CANOE_ST_030_SEATBELT_CONTEXT_ADJUST	214	의도하지 않은
ST_031	TC_CANOE_ST_031_DISTANCE_DISPLAY_CONSISTENCY	222	emergency 거리
ST_032	TC_CANOE_ST_EXT_032_USER_SETTING_CHANGE	215	사용자 표시/볼
ST_033	TC_CANOE_ST_EXT_033_HISTORY_QUERY	222 + historyQuery(0)	emergency warn
ST_034	TC_CANOE_ST_034_DUPLICATE_POPUP_GUARD	12	안정된 warning
ST_035	TC_CANOE_ST_035_TIMEOUT_CLEAR_RESTORE	35	timeout-clear 경
ST_036	TC_CANOE_ST_036_FAILSAFE_RECOVERY_STABILITY	201	fail-safe recover
ST_037	TC_CANOE_ST_037_AUDIO_CHANNEL_STABILITY	215	활성 emergency
ST_038	TC_CANOE_ST_038_VISUAL_CHANNEL_STABILITY	220	시각 우선 warn
ST_045	TC_CANOE_ST_EXT_045_TRIP_SEQUENCE	200	전체 주행 시퀀
ST_046	TC_CANOE_ST_EXT_046_FAILSAFE_RECOVERY	201	fail-safe recover

0.14.15 Wave 2 완료 업데이트 (UT_004/UT_005)

- UT_004: TC_CANOE_UT_CORE_004_V2X_EVENT_MAINTAIN은 scenario 9를 사용하며 V2X.can의 ETA/source 우선순위 처리를 검증합니다.
- UT_005: TC_CANOE_UT_CORE_005_ADAS_DECEL_ASSIST는 scenario 16를 사용하며 ADAS.can의 proximity-risk 및 decel-assist 활성화를 검증합니다.
- 위 추가분까지 포함하면 현재 direct-ownership 05 core 기준선 (UT_003, UT_004, UT_005, UT_011, UT_014, UT_015, UT_063, UT_064, UT_076, UT_077)은 실행 가능한 asset contract를 확보한 상태입니다.

0.14.16 Wave 5 gateway 및 extension UT 기준선

- UT_001: TC_CANOE_UT_CORE_001_CGW_CHS_GW는 scenario 216, 217, 218, 219를 사용해 drive, speed, steering, brake 맥락에 대한 정규화된 chassis-state 전달을 검증합니다.
- UT_002: TC_CANOE_UT_CORE_002_CGW_INFOTAINMENT_GW는 scenario 2, 7, 8을 사용해 infotainment gateway 경로를 통한 zone, direction, distance, speed-limit 전달을 검증합니다.
- UT_009: TC_CANOE_UT_CORE_009_NAV_CTX_MGR는 scenario 2, 3, 7, 8을 사용해 zone 맥락 분류와 navigation 맥락 안정화를 검증합니다.
- UT_012: TC_CANOE_UT_CORE_012_BODY_GW_ROUTE는 scenario 208, 4를 사용해 최종 warning result가 body 출력 경로로 ambient-route 전달되는지 검증합니다.
- UT_013: TC_CANOE_UT_CORE_013_IVI_GW_ROUTE는 scenario 209, 30, 33을 사용해 최종 warning result가 IVI/cluster 출력 경로로 cluster-route 전달되는지 검증합니다.
- UT_016: TC_CANOE_UT_EXT_016_CHS_BRAKE_EXT는 scenario 216, 219를 사용해 brake 및 parked-state 맥락 전파를 검증합니다.
- UT_017: TC_CANOE_UT_EXT_017_CHS_DYNAMICS_EXT는 scenario 225를 사용해 suspension/chassis-dynamics 맥락 전파를 검증합니다.
- UT_018: TC_CANOE_UT_EXT_018_BODY_ENTRY_EXIT는 scenario 226, 227을 사용해 door 및 tailgate 진입/이탈 맥락 전파를 검증합니다.
- UT_019: TC_CANOE_UT_EXT_019_BODY_OCCUPANT_PROTECTION는 scenario 228을 사용해 occupant-detection 및 occupant-protection 맥락 전파를 검증합니다.

0.14.17 Wave 6 service, assist, router UT 기준선

- UT_020: TC_CANOE_UT_EXT_020_BODY_COMFORT는 scenario 229, 230을 사용해 HVAC remote-climate 동작, AFLS/AHLS comfort-lighting 전파, static seat/sunroof comfort 출력을 함께 검증합니다.
- UT_021: TC_CANOE_UT_EXT_021_IVI_DISPLAY_SERVICE는 scenario 229, 30, 215를 사용해 TMU service 전달, HUD 표시 렌더링, AMP audio-policy 전파를 검증합니다.
- UT_022: TC_CANOE_UT_EXT_022_IVI_SERVICE_ACCESS는 scenario 229를 사용해 TMU service 상태와 digital-key service-access 전파를 검증합니다.
- UT_023: TC_CANOE_UT_EXT_023_ADAS_DRIVE_ASSIST는 scenario 233을 사용해 FCAM state 기반의 LDWS_LKAS, RPC drive-assist / road-preview 전파를 검증합니다.
- UT_024: TC_CANOE_UT_EXT_024_ADAS_PARKING_PERCEPTION는 scenario 234를 사용해 AVM / ultrasonic / SPAS / RSPA state 기반의 PKM, SPM parking-perception 전파를 검증합니다.
- UT_025: TC_CANOE_UT_EXT_025_WARNING_DELIVERY_BOUNDARY는 scenario 18, 203, 204를 사용해 fail-safe 진입과 SGW/DCM delivery-boundary, response-state 전파를 함께 검증합니다.
- UT_026: TC_CANOE_UT_EXT_026_DOMAIN_ROUTER_PROPULSION는 scenario 232를 사용해 EOP, MCU, INVERTER를 통한 propulsion 맥락 전파를 검증합니다.
- UT_027: TC_CANOE_UT_EXT_027_DOMAIN_ROUTER_POWER_CHARGE는 scenario 231를 사용해 OBC, DCDC를 통한 charging 맥락 전파를 검증합니다.

0.14.18 Wave 4 integration 기준선

공식 범위	Native Asset
IT_002	TC_CANOE_IT_CORE_002_SCHOOLZONE_PATH
IT_003	TC_CANOE_IT_CORE_003_HIGHWAY_NOSTEER_PATH
IT_001	TC_CANOE_IT_CORE_001_BASE_ACTIVATION
IT_004	TC_CANOE_IT_V2_004_POLICE_RX
IT_005	TC_CANOE_IT_V2_005_AMBULANCE_RX
IT_006	TC_CANOE_IT_V2_006_ARBITRATION

공식 범위	Native Asset
IT_007	TC_CANOE_IT_CORE_007_AMBIENT_OUTPUT
IT_008	TC_CANOE_IT_CORE_008_CLUSTER_DIRECTION_OUTPUT
IT_009	TC_CANOE_IT_V2_009_TIMEOUT_CLEAR
IT_010	TC_CANOE_IT_V2_010_DECEL_ASSIST
IT_011	TC_CANOE_IT_V2_011_FAILSAFE_MIN_WARNING
IT_012	TC_CANOE_IT_EXT_012_OBJECT_RISK_EVENTLOG
RET_IT_013	TC_CANOE_IT_EXT_002_DRIVER_CONTEXT
IT_013	TC_CANOE_IT_013_SEATBELT_CONTEXT
IT_014	TC_CANOE_IT_014_DISPLAY_POLICY
IT_015	TC_CANOE_IT_015_TURN_LAMP_CONTEXT
IT_016	TC_CANOE_IT_016_DRIVE_MODE_SENSITIVITY
IT_017	TC_CANOE_IT_017_AUDIO_VOLUME_POLICY
IT_018	TC_CANOE_IT_EXT_018_EMERGENCY_PLUS_TTC
RET_IT_015	TC_CANOE_IT_BASE_001_POWERTRAIN_STATE
IT_019	TC_CANOE_IT_019_POWERTRAIN_PARKED_BASELINE
IT_020	TC_CANOE_IT_020_POWERTRAIN_DRIVE_BASELINE
RET_IT_016	TC_CANOE_IT_BASE_002_CHASSIS_STATE
IT_021	TC_CANOE_IT_021_CHASSIS_STEERING_BASELINE
IT_022	TC_CANOE_IT_022_CHASSIS_BRAKE_BASELINE
IT_023	TC_CANOE_IT_023_CHASSIS_ACCEL_BASELINE
RET_IT_017	TC_CANOE_IT_BASE_024_BODY_STATE
IT_024	TC_CANOE_IT_BASE_024_BODY_STATE
IT_025	TC_CANOE_IT_EXT_025_WINDOW_STATE
IT_026	TC_CANOE_IT_BASE_026_BASIC_DISPLAY_UI
RET_IT_019	TC_CANOE_IT_BASE_027_COMFORT_CONTEXT / TC_CANOE_IT_BASE_030_BODY_SECURITY_CONTE
IT_027	TC_CANOE_IT_BASE_027_COMFORT_CONTEXT
IT_028	TC_CANOE_IT_EXT_028_BODY_CONTROL_LOCK
IT_029	TC_CANOE_IT_EXT_029_WIPER_RAIN_BASELINE
IT_030	TC_CANOE_IT_BASE_030_BODY_SECURITY_CONTEXT
IT_031	TC_CANOE_IT_031_AUDIO_GUIDE_RUNTIME
RET_IT_021	TC_CANOE_IT_EXT_004_OUTPUT_AVAILABILITY
IT_032	TC_CANOE_IT_032_OUTPUT_FALLBACK
IT_033	TC_CANOE_IT_033_DUPLICATE_POPUP_SUPPRESSION
IT_034	TC_CANOE_IT_034_CHANNEL_RESTORE
IT_035	TC_CANOE_IT_EXT_035_DISTANCE_HISTORY

0.15 실행 가이드

원문:

- ../../verification/execution-guide.md

동기화 기준:

- 5d83ee7f
- technical identifier, test asset ID, SysVar 이름은 원문과 동일하게 유지합니다.

Important

이 문서는 현재 개발 baseline과 계획 중인 target architecture를 반영합니다. runtime, diagnostic, verification 세부사항 가운데 일부는 아직 구현 중이며 변경될 수 있습니다.

0.15.1 목적

이 문서는 현재 native CANoe SIL 검증 자산의 실행 흐름을 정의합니다.

다음 항목에 대한 공식 실행 기준 문서입니다.

- Test Unit 등록과 실행
- TEST_SCN, TEST_BAS를 통한 harness 상호작용
- native execution 이후 evidence 수집

0.15.2 검증 모델

현재 baseline은 harness-first execution model을 사용합니다.

요소	역할
TEST_SCN	scenario stimulus를 주입하고 scenario 단위 verdict 상태를 설정합니다.
TEST_BAS	Test::base* summary variable을 통해 baseline 결과를 집계합니다.
native CANoe Test Unit asset	선택한 verification package를 CANoe 내부에서 실행합니다.

0.15.3 현재 active native asset

현재 active native execution baseline으로 유지하는 자산은 아래 두 개입니다.

1. canoe/tests/modules/test_units/TC_CANOE_UT_CORE_011_ADAS_WARNING_SELECTION
2. canoe/tests/modules/test_units/TC_CANOE_IT_V2_011_FAILSAFE_MIN_WARNING

이 자산은 현재 baseline을 뜻할 뿐이며, 미래 전체 test architecture가 완성되었다는 의미는 아닙니다.

0.15.4 Harness contract

현재 실행 흐름은 아래 harness variable에 의존합니다.

변수	의미
Test::scenarioCommand	one-shot execution trigger
Test::scenarioCommandAck	accepted command identifier
Test::scenarioResult	TEST_SCN이 기록하는 scenario 단위 verdict
Test::baseScenarioId	baseline aggregation에 사용하는 scenario identifier
Test::baseScenarioResult	TEST_BAS가 기록하는 baseline PASS/FAIL 결과
Test::baseFlowCoverageMask	baseline coverage summary

변수	의미
Test::baseTraceSnapshotId	baseline trace anchor
Test::baseTestHealth	baseline harness health summary

0.15.5 표준 실행 절차

1. CANoe GUI에서 active configuration을 엽니다.
2. 필요한 runtime node, database, panel asset, SysVar surface가 모두 준비되었는지 확인합니다.
3. CANoe Test Unit 환경에 대응하는 *.vtestunit.yaml 자산을 등록합니다.
4. GUI bulk import가 필요하면 canoe/tests/modules/test_units/assign/ 아래 wrapper folder를 사용합니다.
5. measurement를 시작합니다.
6. 선택한 native Test Unit을 실행합니다.
7. scenario 단위 verdict와 baseline summary verdict를 모두 확인합니다.
8. evidence policy가 요구하는 report, screenshot, supporting evidence를 저장합니다.

0.15.6 현재 매핑

자산	주요 scope	주요 runtime chec
TC_CANOE_UT_CORE_011_ADAS_WARNING_SELECTION	school-zone warning-selection path	selectedAlert*, war
TC_CANOE_IT_V2_011_FAILSAFE_MIN_WARNING	boundary fail-safe path	failSafeMode, dece

0.15.7 기대 산출물

공식 native execution은 필요에 따라 아래 산출물을 남겨야 합니다.

- compile-clean 결과
- native test verdict
- native report path
- screenshot 또는 GUI capture
- evidence policy가 요구하는 write-window 또는 trace support

0.15.8 경계

이 문서는 아래 항목을 정의하지 않습니다.

- 최종 customer-facing test architecture
- CANoe native execution surface 밖의 packaging ownership
- 05/06/07 전체 세트의 광범위한 재설계

이 영역은 이후 customer workproduct baseline을 기준으로 다시 설계될 수 있습니다.

0.15.9 개발 메모

현재 가이드는 active CANoe surface에 대해 유효한 execution baseline입니다. 다만 프로젝트 전체 test architecture는 이후 customer 문서 체인을 기준으로 다시 재구성될 수 있습니다.

0.16 검증 증빙 로그 표준 (CANoe SIL)

원문:

- ../../verification/evidence-policy.md

동기화 기준:

- 5d83ee7f
- 원문이 이미 한국어 중심으로 유지되므로 ko/ mirror는 canonical 내용을 그대로 따라갑니다.

Important

이 문서는 현재 개발 baseline과 계획 중인 target architecture를 반영합니다. runtime, diagnostic, verification 세부사항 가운데 일부는 아직 구현 중이며 변경될 수 있습니다.

관련 운영 표준:

- canoe/docs/verification/VECTOR_ALIGNED_CLOSEOUT_STANDARD.md

0.16.1 1) 목적

- 05_Unit_Test.md, 06_Integration_Test.md, 07_System_Test.md에 입력할 증빙 로그 포맷을 단일화합니다.
- 검증 축을 로직(Logic) / 통신(Comm) / 주기(Period)로 분리해 PASS 근거를 명확히 합니다.
- 실측값 없이 PASS가 기록되는 것을 방지합니다.

0.16.2 2) 고정 실행 순서

1. 개발완성도 체크: 컴파일 성공 + 핵심 경로 동작 확인 (Core/Body/Cluster/UiRender)
2. 로그 표준 고정: verification_log.csv 필드 채우기
3. 시간 규칙 채점: 50/100/150/1000ms
4. 패널 캡처: 차량화면 -> 제어패널 -> 상태모니터
5. 05/06/07 반영: Pass/Fail + 증빙 링크 입력

0.16.3 3) 개발완성도 체크 기준 (Step 1)

- 컴파일 에러 0건
- 측정 시작 성공
- scenario 주입 시 아래 파생 출력이 유효값으로 변해야 합니다.
  - Core::selectedAlertLevel, Core::selectedAlertType
  - Body::ambientMode, Body::ambientColor, Body::ambientPattern
  - Cluster::warningTextCode
  - UiRender::renderMode, UiRender::renderColor, UiRender::renderTextCode

0.16.4 4) 증빙 폴더 규칙

- 루트: canoe/logging/evidence/
- 계층:
  - UT/<run_id>/
  - IT/<run_id>/
  - ST/<run_id>/
  - FULL/<run_id>/ (FULL은 회귀 전용이며 공식 closeout seed는 없습니다.)
- 각 run 폴더의 최소 필수 파일:
  - verification_log.csv
  - raw_write_window.txt
  - capture_index.csv
  - captures/

  - native_reports/
  - native_report_manifest.json
  - supplementary/trace/
  - supplementary/logging/

Write Window Drop Root

- 루트: canoe/logging/evidence/incoming/
- tier별 표준 저장 파일:
  - UT/raw_write_window.txt
  - IT/raw_write_window.txt
  - ST/raw_write_window.txt
  - FULL/raw_write_window.txt
- GUI의 Write Window export 기본 경로는 위 경로로 고정합니다.
- post-run 자동화는 위 표준 경로를 우선 수집하고, 기존 canoe/tmp/write_window/는 migration fallback으로만 사용합니다.

Trace and Logging Drop Root

- 루트: canoe/logging/evidence/incoming/
- tier별 표준 supplementary 경로:
  - UT/trace/, IT/trace/, ST/trace/, FULL/trace/
  - UT/logging/, IT/logging/, ST/logging/, FULL/logging/
- CAN/Ethernet trace export, logging block output, BLF/ASC/MF4/PCAP 계열 산출물은 위 경로에 저장합니다.
- post-run은 위 경로를 자동 수집해 run-local supplementary/trace, supplementary/logging으로 복사합니다.

Write Window 키워드 규칙

- 입력 이벤트: [EVIDENCE_IN] scenario=<id> inputTsMs=<ms>
- 출력 이벤트: [EVIDENCE_OUT] scenario=<id> outputTsMs=<ms> result=<0|1> ...
- inputTsMs, outputTsMs를 이용해 latency_ms를 계산합니다.

Seed Source 규칙

- verification_log.csv 초기 row는 수동 템플릿 복사가 아니라 scripts/quality/init_evidence_run.py로 생성합니다.
- seed SoT:
  - driving-alert-workproducts/05_Unit_Test.md
  - driving-alert-workproducts/06_Integration_Test.md
  - driving-alert-workproducts/07_System_Test.md
  - canoe/docs/verification/test-asset-mapping.md
  - canoe/tests/modules/test_units/<asset>/<asset>.can
- expected는 공식 05/06/07 표에서 추출합니다.
- scenario_id는 native CAPL의 launchScenarioAndWait(...) 호출에서 시드합니다.
- rule_type / rule_ms는 공식 표 문장에 대한 deterministic seed이며, 애매한 row는 note에 manual confirmation을 남깁니다.
- FULL은 regression tier이므로 prepare seed 대상이 아니며, collect/post-run으로 native report와 raw log만 저장합니다.

0.16.5 5) verification_log.csv 필수 컬럼

- tier: UT|IT|ST

- test_id
- native_asset
- scenario_id
- input_ts_ms
- output_ts_ms
- latency_ms (비어 있어도 채점기가 계산)
- rule_type: LE|GE|BETWEEN|EQ
- rule_ms: 예) 150, 1000, 1000:1300
- expected
- observed
- logic_verdict: PASS|FAIL
- comm_verdict: PASS|FAIL
- verdict: 수동 입력값(선택)이며, 최종 판정은 채점기 결과(computed_verdict)를 기준으로 합니다.
- owner
- run_date: YYYY-MM-DD
- evidence_log_path
- evidence_capture_path
- note

관찰 문자열(**observed**) 표준

- observed는 현재 [EVIDENCE_OUT]의 핵심 필드를 key/value 문자열로 저장합니다.
- 기본 포함 필드:
  - level, type, code, timeout, risk, decel, failSafe, renderType, renderCode
- 선택 포함 필드:
  - release, objValid, objClass, objTtc, objEvent

0.16.6 6) 판정 규칙

6.1 주기(Period)

- LE: latency_ms <= rule_ms
- GE: latency_ms >= rule_ms
- BETWEEN: min <= latency_ms <= max
- EQ: latency_ms == rule_ms

6.2 로직(Logic)

- 로직 기대값(expected)과 실제 상태(observed)를 비교해 logic_verdict를 기록합니다.

6.3 통신(Comm)

- 기대 경로의 파생 출력 반영 여부를 comm_verdict로 기록합니다.
- 예: Core -> Body -> UiRender, Core -> Cluster -> UiRender

6.4 최종 판정

- computed_verdict = PASS 조건:
  - 주기 판정 PASS
  - logic_verdict=PASS
  - comm_verdict=PASS
  - owner, run_date, evidence_*가 비어 있지 않을 것
- 위 조건 중 하나라도 위반하면 FAIL입니다.

0.16.7 7) 표준 시간 기준

- 50ms: 출력 주기
- 100ms: 입력/통신 주기
- 150ms: 즉시 반영 상한(100ms + 50ms)
- 1000ms: timeout clear 기준

0.16.8 8) 캡처 규칙

- 테스트당 최소 3장:
  - {test_id}_01_vehicle.png
  - {test_id}_02_control.png
  - {test_id}_03_monitor.png
- capture_index.csv에 파일 경로와 설명을 반드시 매핑합니다.

0.16.9 9) 실행 명령 예시

```
python scripts/run.py verify prepare --run-id 20260306_1930
python scripts/run.py verify fill-score --tier UT --run-id 20260306_1930 --owner <OWNER>
```

0.16.10 10) 05/06/07 반영 규칙

- computed_verdict를 기준으로 문서 상단 Pass/Fail을 입력합니다.
- evidence_log_path, evidence_capture_path를 테스트 ID와 1:1로 연결합니다.
- 문서 값과 scored report 값이 다르면 문서를 FAIL로 처리합니다.
- trace/log 관련 테스트는 supplementary/trace 또는 supplementary/logging 없이 closeout 완료로 보지 않습니다.

0.16.11 11) 승인 규칙

- 자동화는 candidate verdict까지만 생성합니다.
- 공식 PASS/FAIL 반영 전에는 reviewer approval이 필요합니다.
- FAIL 또는 ERROR는 templates/fail_disposition_template.md 기준 disposition을 남깁니다.

S4. 진단 및 관측성 세트

본 별첨은 서비스 상태와 경계 조건을 독립적으로 관측하는 진단·관측 구조를 보강한다.

원문 기준 경로

- canoe/docs/Kor/contracts/diagnostic-matrix.md
- canoe/docs/Kor/contracts/diagnostic-sysvar-contract.md
- canoe/docs/Kor/verification/diagnostic-seam-design.md

0.17 Diagnostic Matrix

원문:

- ../../contracts/diagnostic-matrix.md

동기화 기준:

- 5d83ee7f
- request/response frame 이름, SID, DID/DTC, coverage tier 이름은 canonical technical string 으로 유지합니다.

Important

이 문서는 CANoe SIL baseline에서 공식 verdict를 설명하기 위해 필요한 최소 diagnostic matrix를 정의합니다. diagnostic seam, tester flow, service-state contract가 고정되기 전까지 일부 항목은 변경될 수 있습니다.

0.17.1 1. 목적

이 문서는 현재 프로젝트 baseline에서 반드시 유지해야 하는 최소 diagnostic matrix를 정의합니다.

이 matrix는 의도적으로 작게 유지합니다.

대상은 현재 official verdict를 강하게 설명하기 위해 diagnostic 근거가 꼭 필요한 항목만입니다.

0.17.2 2. 최소 matrix 필드

현재 minimum matrix 필드는 다음과 같습니다.

1. ECU
2. ReqFrame
3. RespFrame
4. SID
5. DID/DTC
6. PositiveResp
7. NegativeResp
8. Timeout
9. SourceBus
10. TargetBus
11. CoverageTier

이 필드만으로도 다음을 설명할 수 있습니다.

- diagnostic interaction의 owner가 누구인지
- request/response path를 어디에서 관찰하는지
- 현재 verdict가 positive, negative, timeout을 구분해야 하는지
- 어떤 bus boundary가 개입되는지

- 현재 diagnostic 설명 깊이가 어느 tier까지 필요한지

0.17.3 3. 현재 공식 matrix

Source ID	ECU	ReqFrame	RespFrame
UT_063	SGW	TBD_DIAG_REQ_SGW_SECURITY	TBD_DIAG_RESP_SGW_SECURI
UT_064	DCM	TBD_DIAG_REQ_DCM_STATE	TBD_DIAG_RESP_DCM_STATE
IT_040	SGW + DCM + runtime owner	TBD_DIAG_REQ_SERVICE_SECURITY	TBD_DIAG_RESP_SERVICE_SEC
ST_043	system-level diagnostic context	TBD_DIAG_REQ_SYSTEM_CONTEXT	TBD_DIAG_RESP_SYSTEM_CON

0.17.4 4. 필드 해석 규칙

1. ReqFrame과 RespFrame은 정확한 runtime path가 고정되기 전까지 TBD_* 상태로 둘 수 있습니다.
2. SID와 DID/DTC는 첫 concrete tester flow가 승인되기 전까지 TBD 상태로 둘 수 있습니다.
3. PositiveResp, NegativeResp, Timeout은 verdict가 해당 response class를 구분해야 하는지를 나타냅니다.
4. CoverageTier는 diagnostic 설명이 어느 수준까지 내려가야 하는지를 표현합니다.

권장 tier 의미는 다음과 같습니다.

- Tier-1
  - 단일 ECU 또는 단일 state 해석 수준
- Tier-2
  - integrated runtime과 ownership 해석 수준
- Tier-3
  - system-level verdict explanation 수준

0.17.5 5. 향후 심화 필드

minimum matrix가 안정화된 뒤에는 다음 필드를 추가할 수 있습니다.

1. session control
2. security access
3. NRC policy
4. gateway route ownership
5. ECU DID catalog
6. ODX/CDD-based tester interpretation

이 항목들은 현재 entry criteria는 아닙니다.

추후 diagnostic 심화 단계에서만 추가합니다.

0.17.6 6. 관련 문서

이 문서는 다음 문서와 함께 사용합니다.

- diagnostic-description.md
- diagnostic-sysvar-contract.md
- ../verification/diagnostic-coverage.md
- ../verification/diagnostic-seam-design.md

0.18 Diagnostic SysVar Contract

원문:

- ../../contracts/diagnostic-sysvar-contract.md

동기화 기준:

- 5d83ee7f
- Diag::* 이름과 개별 SysVar 식별자는 canonical technical string으로 유지합니다.

Important

이 문서는 Diag::* system variable이 diagnostic observation contract로 어떻게 사용되는지를 정의합니다. runtime path와 diagnostic seam 구현이 바뀌면 이 문서도 함께 갱신해야 합니다.

0.18.1 1. 목적

이 문서는 Diag::* namespace로 노출되는 stable diagnostic observation contract를 정의합니다.

목적은 diagnostic request/response evidence를 다음 소비자에게 안정적으로 노출하는 것입니다.

- verification logic
- trace reviewer
- panel 또는 tool observer

단, 제품 runtime을 임시 debug wiring으로 오염시키지 않는 범위에서만 사용합니다.

0.18.2 2. 범위

이 계약은 다음 active namespace를 대상으로 합니다.

- project/sysvars/project.sysvars 안의 Diag

이 문서는 의미와 사용 규칙을 정의하며, 개별 UDS service의 상세 payload를 모두 설명하지는 않습니다.

0.18.3 3. Namespace baseline

현재 active diagnostic namespace는 Diag 입니다.

여기에는 request mirror, response mirror, counter, timestamp, verdict-facing semantic seam이 포함됩니다.

0.18.4 4. Request-side contract

SysVar	의미	전형적 producer
Diag::LastRequestTarget	가장 최근 diagnostic request의 target ECU 또는 service code	diagnostic tester
Diag::LastRequestSid	가장 최근 request의 service identifier	diagnostic tester
Diag::LastRequestDidHigh	가장 최근 request의 DID high byte	diagnostic tester
Diag::LastRequestDidLow	가장 최근 request의 DID low byte	diagnostic tester
Diag::LastRequestSourceBus	request가 발생한 source bus code	diagnostic tester
Diag::RequestCounter	누적 request count	diagnostic tester
Diag::LastRequestTimeMs	가장 최근 request timestamp(ms)	diagnostic tester

0.18.5 5. Response-side contract

SysVar	의미	전형적 produ
Diag::LastResponseTarget	가장 최근 diagnostic response의 target ECU 또는 service code	diagnostic res
Diag::LastResponseCode	가장 최근 response의 response code	diagnostic res
Diag::LastResponseData0	evidence 요약용 첫 번째 response payload byte	diagnostic res
Diag::LastResponseData1	evidence 요약용 두 번째 response payload byte	diagnostic res
Diag::LastResponseOk	가장 최근 response의 positive/negative flag	diagnostic res
Diag::LastResponseSourceBus	response가 발생한 source bus code	diagnostic res
Diag::ResponseCounter	누적 response count	diagnostic res
Diag::LastResponseTimeMs	가장 최근 response timestamp(ms)	diagnostic res

0.18.6 6. Verdict-facing seam contract

SysVar	의미	전형적 pro
Diag::SecurityState	verification용 현재 security interpretation	diagnostic/
Diag::ServiceState	verification용 현재 service availability interpretation	diagnostic/
Diag::RouteOwner	가장 최근 diagnostic-linked verdict의 active route ownership interpretation	gateway/ru
Diag::ResponseKind	가장 최근 diagnostic-linked verdict의 semantic response class	diagnostic
Diag::ReasonCode	diagnostic-linked decision의 compact verdict-facing reason code	diagnostic/

이 변수들은 verdict 설명을 위한 semantic seam입니다.

이 변수들만으로 full transport trace나 full tester payload review를 대체할 수는 없습니다.

0.18.7 7. 계약 규칙

7.1 Diag::*는 observation contract입니다 Diag::*는 diagnostic runtime에서 실제로 발생한 내용을 관찰 가능하게 만드는 mirror입니다.

diagnostic 의미를 Diag::* 안에만 고정해두면 안 됩니다.

제품 동작 자체는 여전히 실제 diagnostic runtime path에서 구현되어야 합니다.

7.2 Counter는 누적값입니다 RequestCounter와 ResponseCounter는 active session/runtime scope의 cumulative mirror입니다.

boolean flag처럼 재사용하면 안 됩니다.

7.3 Timestamp 단위는 ms입니다 LastRequestTimeMs와 LastResponseTimeMs는 millisecond timestamp입니다.

tooling과 evidence review 전반에서 이 단위를 유지해야 합니다.

7.4 Bus code 해석은 일관돼야 합니다 LastRequestSourceBus와 LastResponseSourceBus는 producer와 consumer가 같은 code mapping을 공유할 때만 유효합니다.

bus-code enum이 바뀌면 관련 runtime/test helper와 이 문서를 함께 갱신해야 합니다.

7.5 Response summary field는 evidence 보조용입니다 LastResponseData0와 LastResponseData1은 summary mirror입니다.

빠른 evidence 확인이나 smoke validation에는 유용하지만, deeper analysis가 필요할 때 full diagnostic payload trace를 대체하지는 못합니다.

7.6 Verdict-facing seam은 semantic seam입니다 SecurityState, ServiceState, RouteOwner, ResponseKind, ReasonCode는 official verdict를 reviewer-facing 관점에서 읽기 쉽게 만들기 위한 seam입니다.

구현 세부값을 무제한으로 덤프하는 표면으로 쓰면 안 됩니다.

작고 안정적인 semantic vocabulary를 유지하는 것이 원칙입니다.

0.18.8 8. Verification 사용 방식

Diag::*는 다음 용도로 사용합니다.

- request 송신 여부 smoke 확인
- response 수신 여부 smoke 확인
- positive / negative / timeout expectation에 대한 pass/fail gating
- trace, write-window, evidence 간 correlation
- 현재 official diagnostic scope에 대한 verdict-facing explanation

full transport trace가 필요한 상황에서는 Diag::*만을 유일한 증거로 사용하면 안 됩니다.

0.18.9 9. 갱신 규칙

diagnostic runtime path가 바뀌면 다음 순서로 갱신합니다.

1. Diag::* producer/consumer logic 갱신
2. variable surface가 바뀌면 project/sysvars/project.sysvars 갱신
3. 이 contract 문서 갱신
4. 변경된 evidence semantics를 사용하는 verification 문서 갱신

0.19 Diagnostic Seam Design

원문:

- ../../verification/diagnostic-seam-design.md

동기화 기준:

- 5d83ee7f
- reserved seam 이름과 Diag::* identifier는 canonical technical string으로 유지합니다.

Important

이 문서는 diagnostic-linked verification seam의 최소 설계 기준을 정의합니다. CANoe diagnostic hook, sysvar contract, tester interpretation이 고정되면 내용도 함께 조정될 수 있습니다.

0.19.1 1. 목적

이 문서는 현재 official diagnostic scope를 위해 필요한 minimum seam design을 정의합니다.

현재 official scope는 의도적으로 좁게 유지되며 다음 항목으로 제한됩니다.

- UT_063
- UT_064
- IT_040
- ST_043

이 항목들은 normal oracle/evidence path만으로는 부족하고, diagnostic 또는 security state visibility가 더 강하게 필요합니다.

0.19.2 2. 설계 원칙

1. 모든 internal state가 아니라 verdict-critical state만 노출합니다.
2. business semantics와 transport detail을 분리합니다.
3. seam 값은 CANoe native report, sysvar snapshot, write-window evidence에서 읽기 쉬워야 합니다.
4. 가능하면 UT -> IT -> ST로 올라가도 같은 seam 의미를 유지합니다.
5. official verdict scope가 바뀌지 않는 한 diagnostic seam을 일반 No 항목으로 확장하지 않습니다.

0.19.3 3. 최소 seam field

minimum seam set은 다음 질문에 답할 수 있어야 합니다.

- 현재 어떤 security 또는 diagnostic state가 active인지
- 어떤 route owner 또는 gateway owner가 active인지
- 마지막 request/response class가 무엇이었는지
- verdict가 timeout, denial, unavailable service state 때문에 막혔는지

권장 minimum field:

- diag_state
- security_state
- service_state
- route_owner
- last_sid
- last_did_or_dtc
- response_kind
- timeout_flag
- source_bus

- target_bus
- reason_code

위 이름은 semantic field 이름이며, 최종 implementation identifier를 강제하는 것은 아닙니다.

0.19.4 4. 현재 0304-aligned seam candidate

현재 system-variable baseline은 0304_System_Variables.md와 Diag::*를 통해 필요한 seam의 일부를 이미 노출하고 있습니다.

먼저 다음 기존 variable을 우선 활용합니다.

Semantic need	Current candidate sysvar surface
fail-safe state	Core::failSafeMode
path health	CoreState::warningPathStatus, CoreState::e2eHealthState
route and boundary condition	Powertrain::RoutingPolicy, Powertrain::BoundaryStatus, Body::Bo
diagnostic request identity	Diag::LastRequestTarget, Diag::LastRequestSid, Diag::LastRequest
diagnostic response identity	Diag::LastResponseTarget, Diag::LastResponseCode, Diag::LastRes
ECU-local diagnostic state hint	Chassis::ChassisDiagState, Body::BodyDiagState, Infotainment::Inf
ECU-local diagnostic request/response hint	Chassis::ChsDiagServiceId, Chassis::ChsDiagRespCode, Body::Bcm

이 변수들만으로도 skeleton wiring과 evidence capture는 시작할 수 있습니다.

다만 모든 official diagnostic verdict를 깔끔하게 설명하기에는 아직 부족합니다.

0.19.5 5. 현재 reserved seam addition

다음 semantic field는 verification용 Diag::* 아래에 reserved seam으로 유지합니다.

producer wiring은 아직 진행 중일 수 있습니다.

Reserved seam	필요한 이유
Diag::SecurityState	UT_063, IT_040, ST_043는 indirect interpretation만으로는 부족하고 stable security ver
Diag::ServiceState	integrated service-context verdict를 mixed runtime side effect만으로 판단하면 설명력이
Diag::RouteOwner	현재 route/boundary health 표면만으로는 ownership explanation이 충분히 강하지 않습
Diag::ResponseKind	Positive / Negative / Timeout / Unavailable를 verdict-facing 의미로 안정적으로 해석해야
Diag::ReasonCode	최종 official verdict에서 diagnostic-linked decision의 이유를 짧고 안정적으로 설명해야

0.19.6 6. 남아 있는 meaning-level gap

지금 남은 gap은 variable reservation 자체가 아닙니다.

핵심은 producer-side semantic wiring입니다.

Remaining gap	필요한 이유
semantic producer wiring	runtime 또는 harness logic이 reserved seam을 일관되게 써줘야 합니다
stable enum meaning	UT, IT, ST 전 구간에서 seam 값의 해석이 동일해야 합니다
trace correlation rule	seam update가 Diag::* request/response mirror와 write-window evidence와 연동되

0.19.7 7. Baseline enum contract

UT부터 ST까지 one stable numeric interpretation을 사용합니다.

Seam	Value	Meaning
Diag::SecurityState	0	unavailable or inactive
Diag::SecurityState	1	session open or degraded security context
Diag::SecurityState	2	authorized and route-usable
Diag::SecurityState	3	denied, faulted, or fail-safe-blocked
Diag::ServiceState	0	unavailable
Diag::ServiceState	1	degraded
Diag::ServiceState	2	available
Diag::RouteOwner	0	none or unknown
Diag::RouteOwner	1	SGW
Diag::RouteOwner	2	DCM
Diag::RouteOwner	3	integrated warning path
Diag::ResponseKind	0	none
Diag::ResponseKind	1	positive
Diag::ResponseKind	2	negative
Diag::ResponseKind	3	timeout
Diag::ResponseKind	4	unavailable or denied
Diag::ReasonCode	0	none
Diag::ReasonCode	1	boundary down
Diag::ReasonCode	2	fail-safe active
Diag::ReasonCode	3	service degraded
Diag::ReasonCode	4	policy denied
Diag::ReasonCode	5	response timeout
Diag::ReasonCode	6	route mismatch

0.19.8 8. 현재 official scope별 seam design

Source ID	Seam question
UT_063	security preset이 의도한 security state와 ownership context로 해석됐는가
UT_064	diagnostic preset이 의도한 diagnostic state와 request class로 해석됐는가
IT_040	integrated service, security, diagnostic context가 기대한 runtime verdict로 수렴하는가
ST_043	system-level service/security/diagnostic context가 최종 scenario verdict를 정당하게 설명하는가

0.19.9 9. Planned assertion profile

Source ID	Planned nominal assert set
UT_063	SecurityState=2, RouteOwner=3 or 1
UT_064	LastRequestSid=0x22, ResponseKind=1, ReasonCode=0
IT_040	ServiceState=2, SecurityState=2, RouteOwner=3, ResponseKind=1, ReasonCode=0
ST_043	scenario verdict remains explainable with ServiceState=2, SecurityState=2, RouteOwner=3, ReasonC

0.19.10 10. Publisher ownership baseline

reserved seam은 producer를 하나로 제한합니다.

Producer	Owned reserved seam
SGW.can	Diag::SecurityState, Diag::RouteOwner
DCM.can	Diag::ServiceState, Diag::ResponseKind, Diag::ReasonCode, Diag::LastRequestSid, Diag::LastRespons

0.19.11 11. ST_043 scenario stimulus contract

첫 executable ST_043 scenario는 세 phase만 사용합니다.

1. nominal context
   - boundary healthy
   - fail-safe inactive
   - service available
   - expected seam: ServiceState=2, SecurityState=2, RouteOwner=3, ReasonCode=0
2. degraded context
   - emergency refresh intentionally degraded
   - warningPathStatus=1, failSafeMode=1
   - expected seam: ServiceState=1, SecurityState=3, RouteOwner=1, ReasonCode=3
3. blocked context
   - explicit fail-safe active
   - expected seam: ServiceState=1, SecurityState=3, RouteOwner=1, ReasonCode=2

system-level testcase는 visible behavior와 seam explanation이 세 phase 모두에서 일치할 때만 pass로 봅니다.

0.19.12 12. Response interpretation baseline

첫 implementation baseline은 다음 네 종류만 구분합니다.

- positive completion
- negative completion
- timeout
- unavailable service or denied state

response_kind에 권장하는 semantic value:

- Positive
- Negative
- Timeout
- Unavailable

현재 official coverage가 안정화되기 전에는 full NRC catalog로 확장하지 않습니다.

0.19.13 13. 구현 순서

1. UT_063
   - Diag::SecurityState wiring
   - Diag::RouteOwner wiring
2. UT_064
   - ECU-local diagnostic state interpretation wiring
   - Diag::ResponseKind wiring
3. IT_040
   - Diag::ServiceState wiring

- shared ownership interpretation 재사용

4. ST_043
   - system-level diagnostic-context verdict seam 연결

0.19.14 14. Expansion gate

다음 중 하나가 true가 될 때만 이 seam set을 확장합니다.

1. official verdict가 session 또는 security access stage에 직접 의존할 때
2. reviewer가 SID, DID, DTC, NRC 수준 설명을 요구할 때
3. gateway route ownership이 formal acceptance의 일부가 될 때
4. ODX/CDD-based tester interpretation이 현재 baseline에 필요해질 때

0.19.15 15. 관련 verification 문서

이 문서는 다음 문서와 함께 사용합니다.

- diagnostic-coverage.md
- test-asset-mapping.md
- oracle.md
- diagnostic-sysvar-contract.md